

# EdgeFlow 用户手册

边缘计算平台使用说明书

版本：v0.8.0

编制日期：2026 年 8 月 26 日

适用产品：EdgeFlow v0.8.0

本文档随产品发布，内容与 v0.8.0 开发进展一致；

未实现功能已标注”即将上线”或列入附录 E 已知限制。

版本信息

| 项目   | 内容                            |
|------|-------------------------------|
| 产品名称 | EdgeFlow（边缘计算平台）              |
| 手册版本 | v0.8.0                        |
| 产品版本 | v0.8.0（2026-08-26 发布）         |
| 编制日期 | 2026 年 8 月 25 日               |
| 适用读者 | 平台安装运维人员、边缘节点操作人员、应用部署人员      |
| 编译方式 | XeLaTeX（ctex 文档类），见 README.md |
| 变更记录 | 见附录 D                         |

目录

|                                        |    |
|----------------------------------------|----|
| 版本信息                                   | 2  |
| 第一章 产品概述                               | 7  |
| 1.1 EdgeFlow 是什么                       | 7  |
| 1.2 核心组件                               | 7  |
| 1.3 典型应用场景                             | 8  |
| 1.4 版本与发布（v0.7.0）                      | 9  |
| 1.5 术语约定                               | 10 |
| 1.6 功能边界与已知限制                          | 10 |
| 1.7 手册内容导航                             | 11 |
| 第二章 环境要求与安装部署                          | 12 |
| 2.1 环境要求                               | 12 |
| 2.2 获取软件                               | 12 |
| 2.2.1 方式一：源码构建                         | 12 |
| 2.2.2 方式二：使用发布制品                       | 12 |
| 2.2.3 方式三：容器镜像                         | 13 |
| 2.3 安装云端（cloudcore）                    | 13 |
| 2.3.1 方式一：keadm 生成 Kubernetes 部署产物（推荐） | 13 |
| 2.3.2 方式二：Helm 部署                      | 14 |
| 2.3.3 方式三：裸进程运行（单机联调）                  | 15 |

|            |                                                  |           |
|------------|--------------------------------------------------|-----------|
| 2.4        | 接入边缘节点 (edgecore)                                | 15        |
| 2.5        | 部署验证                                             | 16        |
| 2.6        | 卸载与清理                                            | 17        |
| 2.7        | 升级与回滚                                            | 17        |
| 2.8        | 常见问题                                             | 18        |
| <b>第三章</b> | <b>快速入门</b>                                      | <b>19</b> |
| 3.1        | 准备工作                                             | 19        |
| 3.2        | 第一步：构建制品                                         | 19        |
| 3.3        | 第二步：启动云端 cloudcore                               | 19        |
| 3.4        | 第三步：启动边缘 edgecore                                | 20        |
| 3.5        | 第四步：验证节点注册                                       | 20        |
| 3.6        | 第五步：下发第一个 Pod (nginx)                            | 20        |
| 3.7        | 第六步：查看容器与 Pod 状态                                 | 21        |
| 3.8        | 第七步：查看设备数据                                       | 21        |
| 3.9        | 第八步：下发设备指令                                       | 21        |
| 3.10       | 体验模型发布 (可选, v0.7.0)                              | 21        |
| 3.10.1     | 第一步：注册模型                                         | 22        |
| 3.10.2     | 第二步：登记版本                                         | 22        |
| 3.10.3     | 第三步：激活版本                                         | 22        |
| 3.10.4     | 第四步：全量发布                                         | 22        |
| 3.10.5     | 第五步：查询部署影子                                       | 22        |
| 3.11       | 一键 Demo (可选)                                     | 23        |
| 3.12       | 清理                                               | 23        |
| 3.13       | 常见问题                                             | 24        |
| <b>第四章</b> | <b>云端操作指南</b>                                    | <b>25</b> |
| 4.1        | 通用约定                                             | 25        |
| 4.2        | 端点总览                                             | 25        |
| 4.3        | 健康检查                                             | 28        |
| 4.3.1      | GET /healthz                                     | 28        |
| 4.4        | 节点 API                                           | 28        |
| 4.4.1      | GET /api/v1/nodes ——全部节点 (运行视角)                  | 28        |
| 4.4.2      | GET /api/v1/nodes/{nodeID} ——单节点详情               | 29        |
| 4.4.3      | GET /api/v1/edgenodes ——全部节点 (CRD 视角)            | 29        |
| 4.4.4      | GET /api/v1/edgenodes/{nodeID} ——单节点 EdgeNode 对象 | 30        |
| 4.5        | Pod API                                          | 30        |
| 4.5.1      | GET /api/v1/pods ——全部 Pod 状态                     | 30        |
| 4.5.2      | GET /api/v1/nodes/{nodeID}/pods ——单节点 Pod 状态     | 30        |
| 4.6        | 设备 API                                           | 31        |

|            |                                                               |           |
|------------|---------------------------------------------------------------|-----------|
| 4.6.1      | GET /api/v1/devices ——全部设备状态 . . . . .                        | 31        |
| 4.6.2      | GET /api/v1/nodes/{nodeID}/devices ——单节点设备状态 . . . . .        | 31        |
| 4.7        | 下发类 API (可靠下发) . . . . .                                      | 31        |
| 4.7.1      | POST /api/v1/nodes/{nodeID}/podsync ——下发 Pod 配置 . . . . .     | 31        |
| 4.7.2      | POST /api/v1/nodes/{nodeID}/config-sync ——下发配置 . . . . .      | 33        |
| 4.7.3      | POST /api/v1/nodes/{nodeID}/device-command ——下发设备指令 . . . . . | 33        |
| 4.7.4      | 八态响应语义 . . . . .                                              | 34        |
| 4.8        | 监控指标 . . . . .                                                | 36        |
| 4.8.1      | GET /metrics . . . . .                                        | 36        |
| 4.9        | OCSF 在线吊销查询 (RFC 6960) . . . . .                              | 36        |
| 4.9.1      | POST /ocsp . . . . .                                          | 36        |
| 4.10       | API 认证 . . . . .                                              | 37        |
| 4.10.1     | 启用认证 . . . . .                                                | 37        |
| 4.10.2     | 调用方式 . . . . .                                                | 37        |
| 4.11       | 模型 API (v0.7.0: 模型仓库 / 版本管理 / 灰度发布) . . . . .                 | 38        |
| 4.11.1     | 对象模型与键空间 . . . . .                                            | 38        |
| 4.11.2     | 端点明细与请求示例 . . . . .                                           | 40        |
| 4.11.3     | 错误语义 . . . . .                                                | 46        |
| 4.11.4     | config-sync 载荷约定 (边缘模型版本感知, 零边缘代码) . . . . .                  | 47        |
| 4.11.5     | 灰度运营建议 . . . . .                                              | 47        |
| 4.11.6     | 三模式行为矩阵 . . . . .                                             | 48        |
| 4.12       | 排障指南 . . . . .                                                | 50        |
| <b>第五章</b> | <b>边缘节点与自治运行</b> . . . . .                                    | <b>51</b> |
| 5.1        | 接入与连接管理 . . . . .                                             | 51        |
| 5.1.1      | 注册 . . . . .                                                  | 51        |
| 5.1.2      | 心跳保活 . . . . .                                                | 51        |
| 5.1.3      | 通道压缩 . . . . .                                                | 52        |
| 5.1.4      | 断线重连 . . . . .                                                | 52        |
| 5.2        | 可靠投递 . . . . .                                                | 52        |
| 5.3        | Edged 应用调谐 . . . . .                                          | 52        |
| 5.3.1      | 多副本 Pod . . . . .                                             | 53        |
| 5.3.2      | 健康自愈与 CrashLoopBackOff . . . . .                              | 53        |
| 5.3.3      | 镜像漂移检测 . . . . .                                              | 53        |
| 5.3.4      | 资源限制与漂移检测 (v0.2.0) . . . . .                                  | 53        |
| 5.4        | 断网自治 . . . . .                                                | 54        |
| 5.5        | edgecore 配置 (环境变量 + 配置文件) . . . . .                           | 54        |
| 5.6        | 配置热重载 . . . . .                                               | 55        |
| <b>第六章</b> | <b>设备接入</b> . . . . .                                         | <b>57</b> |

|            |                                |           |
|------------|--------------------------------|-----------|
| 6.1        | 设备模型：影子                        | 57        |
| 6.2        | 设备指令下发链路                       | 57        |
| 6.3        | Mapper 框架                      | 58        |
| 6.3.1      | Mapper 装配开关 (v0.2.0)           | 58        |
| 6.3.2      | mock_sensor：模拟传感器              | 58        |
| 6.3.3      | Modbus TCP Mapper              | 58        |
| 6.3.4      | 设备命名空间 (v0.2.0)                | 59        |
| 6.3.5      | OPC-UA 接入状态 (v0.3.0 起)         | 59        |
| 6.3.6      | Mapper 的两种工作模式                 | 59        |
| 6.4        | EventBus 数据面与降级                | 60        |
| 6.5        | 设备状态查询                         | 60        |
| 6.6        | 设备接入检查清单                       | 61        |
| <b>第七章</b> | <b>升级与回滚</b>                   | <b>62</b> |
| 7.1        | 机制总览                           | 62        |
| 7.2        | 升级前准备                          | 62        |
| 7.3        | 升级：keadm upgrade               | 62        |
| 7.4        | 回滚：keadm rollback              | 63        |
| 7.5        | 台账查询：keadm ops-ledger          | 64        |
| 7.6        | Helm 路径（云端组件）                  | 64        |
| 7.7        | 云端（cloudcore）升级与回滚（v0.4.0 起语义） | 64        |
| 7.7.1      | v0.4.0：嵌入式 etcd 持久化升级          | 64        |
| 7.7.2      | 备份与恢复 runbook（简化版）             | 65        |
| 7.7.3      | v0.5.0：外部 etcd 模式切换            | 65        |
| 7.7.4      | v0.6.0：外部模式多副本（真多活）            | 65        |
| 7.7.5      | v0.7.0：模型仓库升级（零迁移）             | 66        |
| 7.7.6      | 版本升级路径一览                       | 66        |
| 7.8        | 端到端演练建议                        | 66        |
| <b>第八章</b> | <b>安全与认证</b>                   | <b>68</b> |
| 8.1        | 云边通道 mTLS                      | 68        |
| 8.1.1      | 启用开关                           | 68        |
| 8.1.2      | 使用 keadm 生成 TLS 产物             | 69        |
| 8.1.3      | 设备接入令牌认证                       | 69        |
| 8.2        | 证书管理                           | 69        |
| 8.2.1      | 证书布局                           | 69        |
| 8.2.2      | 证书轮换与吊销                        | 69        |
| 8.2.3      | 在线吊销查询（OCSP）                   | 71        |
| 8.3        | 管理 API Token 认证                | 72        |
| 8.4        | 外部 etcd 安全（v0.5.0 起）           | 72        |

|             |                                                   |           |
|-------------|---------------------------------------------------|-----------|
| 8.4.1       | 明文护栏（默认拒绝）                                        | 73        |
| 8.4.2       | TLS 与 mTLS                                        | 73        |
| 8.4.3       | etcd 侧最小权限                                        | 73        |
| 8.5         | 审计台账                                              | 73        |
| 8.6         | 安全基线小结                                            | 74        |
| <b>第九章</b>  | <b>常见问题与故障排查</b>                                  | <b>75</b> |
| 9.1         | 日志位置                                              | 75        |
| 9.2         | 常见问题速查表                                           | 75        |
| 9.3         | 分层排查指引                                            | 75        |
| 9.3.1       | 云边通道层                                             | 75        |
| 9.3.2       | 容器运行层                                             | 75        |
| 9.3.3       | 设备数据面层                                            | 77        |
| 9.4         | API 错误语义速记                                        | 77        |
| 9.5         | 环境相关的已知边界                                         | 77        |
| <b>附录 A</b> | <b>附录 A 环境变量参考</b>                                | <b>78</b> |
| A.1         | cloudcore 组                                       | 78        |
| A.2         | edgecore 组                                        | 78        |
| A.3         | Modbus 组                                          | 78        |
| A.4         | keadm / 开发脚本组                                     | 78        |
| <b>附录 B</b> | <b>附录 B API 状态码语义</b>                             | <b>82</b> |
| <b>附录 C</b> | <b>附录 C 术语表</b>                                   | <b>84</b> |
| <b>附录 D</b> | <b>附录 D 版本与变更记录</b>                               | <b>86</b> |
| D.1         | v0.1.0（2026-08-14，MVP 首发）                         | 86        |
| D.2         | v0.1.1（2026-08-18，安全加固）                           | 86        |
| D.3         | v0.2.0（2026-08-18，资源调度 + Mapper 开关 + Modbus 命名空间） | 86        |
| D.4         | v0.3.0（2026-08-19，KNOWN-ISSUES 闭环 + OPC-UA 协议栈）   | 87        |
| D.5         | v0.4.0（2026-08-24，云端持久化：嵌入式 etcd）                 | 87        |
| D.6         | v0.5.0（2026-08-24，外部 etcd 模式）                     | 87        |
| D.7         | v0.6.0（2026-08-25，真多活）                            | 88        |
| D.8         | v0.7.0（2026-08-25，模型仓库/版本管理/灰度发布）                 | 88        |
| D.9         | 已实现 vs 即将上线 / 范围外                                 | 88        |
| <b>附录 E</b> | <b>附录 E 已知限制与边界</b>                               | <b>90</b> |

## 第1章 产品概述

本章内容：介绍 EdgeFlow 边缘计算平台（Edge Computing Platform）的定位、核心组件、典型应用场景、版本发布情况与使用边界，帮助操作人员建立整体认识。

### 1.1 EdgeFlow 是什么

EdgeFlow 是一个面向“云—边—设备”三层架构的轻量级边缘计算平台，当前版本为 v0.7.0。它通过云端统一接口，让操作人员可以在云端完成边缘节点的接入与管理、容器应用的下发与运行、设备的接入与远程控制，并实时掌握边缘侧的运行状态。

EdgeFlow 的典型工作方式：边缘节点上的 **edgecore** 主动连接云端 **cloudcore** 并保持长连接；操作人员通过云端 REST API（Representational State Transfer Application Programming Interface）下发期望（desired）配置；边缘节点执行后，将实际状态（reported）上报云端，形成“下发—执行—上报”的闭环。整个链路包括节点注册、容器调谐（Reconcile）、设备数字孪生（Digital Twin）与配置同步等能力。

自 v0.1.0 发布以来，平台能力持续演进：v0.2.0 引入容器资源调度与超卖准入、Mapper 装配开关与 Modbus 设备命名空间；v0.3.0 交付 OPC-UA 协议栈核心；v0.4.0 起云端由纯内存态升级为分级持久化（嵌入式 etcd）；v0.5.0 支持外部 etcd 模式；v0.6.0 实现外部模式多副本真多活；v0.7.0 新增模型仓库、版本管理与灰度发布。

### 1.2 核心组件

EdgeFlow v0.7.0 由以下组件构成：

| 组件                      | 角色       | 职责                                                                                                                                                                                       |
|-------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cloudcore               | 云端控制面    | 对外提供 REST API (默认端口 8080); 通过 CloudHub (WebSocket, 默认端口 10000) 与边缘节点保持长连接, 处理节点注册、心跳、可靠下发与设备指令; 通过嵌入式 (v0.4.0) 或外部 (v0.5.0) etcd 将注册台账与设备期望状态写穿持久化; 内置模型仓库与灰度发布控制器 (v0.7.0)              |
| edgecore                | 边缘运行时    | 部署在边缘节点上, 由五个子模块协作完成云边通道、元数据管理、容器调谐、设备孪生与事件总线 (详见下文)                                                                                                                                     |
| keadm                   | 安装管理 CLI | 生成云端与边缘的部署产物, 支持 <code>init/join/cert/upgrade/rollback/ops-ledger/t</code> 共 9 个子命令 ( <code>cert</code> 含 <code>rotate</code> 轮换与 <code>revoke</code> 吊销; <code>batch</code> 含批量操作与灰度分批) |
| mock-cloudhub           | 联调工具     | 在本地模拟云端 CloudHub, 用于单独调试 <code>edgecore</code> 的注册、心跳与重连行为                                                                                                                               |
| mappers (Mapper 设备接入程序) | 设备接入     | 将物理或模拟设备接入平台: 内置 <code>mock_sensor</code> 模拟温湿度传感器 (设备名 <code>sensor-01</code> ), <code>modbus</code> 支持 Modbus TCP 协议设备 (默认设备名 <code>mb-sensor-01</code> )                              |

其中, `edgecore` 由以下子模块组成:

- **EdgeHub**: 云边通信通道, 与云端 CloudHub 建立 WebSocket 长连接, 负责消息收发;
- **MetaManager**: 元数据管理, 使用 SQLite 将边缘侧元数据持久化落盘;
- **Edged**: 容器调谐, 基于 Docker 运行时保证容器实际状态与期望状态一致;
- **DeviceTwin**: 设备数字孪生, 维护设备期望值 (desired) 与实际上报值 (reported);
- **EventBus**: 事件总线, 基于 MQTT 提供设备遥测与指令的数据面通道。

### 1.3 典型应用场景

- **边缘容器应用部署**: 在云端下发 nginx 等容器镜像, 由边缘节点本地运行, 业务数据不出边缘;
- **设备数据采集与数字孪生**: 温湿度传感器周期上报属性, 云端可随时查看设备实时数据;



- **设备远程控制**: 通过设备指令下发期望值 (如目标温度), 由 Mapper 在边缘执行;
- **边缘节点管理**: 查看节点注册状态、心跳与平台信息, 感知节点在线/离线状态;
- **配置下发**: 将 ConfigMap/Secret 配置同步到边缘节点, 供边缘应用使用。

## 1.4 版本与发布 (v0.7.0)

- **版本**: v0.7.0 (2026-08-25 发布);
- **发布链**: v0.1.0 (2026-08-14, MVP) → v0.1.1 (2026-08-18, 安全加固) → v0.2.0 (2026-08-18, 资源调度、Mapper 开关与 Modbus 命名空间) → v0.3.0 (2026-08-19, KNOWN-ISSUES 闭环与 OPC-UA 协议栈) → v0.4.0 (2026-08-24, 云端嵌入式 etcd 持久化) → v0.5.0 (2026-08-24, 外部 etcd 模式) → v0.6.0 (2026-08-25, 外部模式多副本真多活) → v0.7.0 (2026-08-25, 模型仓库、版本管理与灰度发布);
- **手册沿革**: v0.1.1 (2026-08-16) 为依据仓库代码审计的修订 (吊销闭环、OCSP、设备认证 token 消费、配置文件与热重载、gzip 压缩、keadm 命令修正等), 产品版本基线仍为 v0.1.0; v0.1.2 (2026-08-18) 为 v0.1.1 生产发布准备轮安全加固入册 (/ocsp per-IP 限流与缓存、OCSP 客户端新鲜度校验、CRL 锁降级日志、P2 代码审查遗留闭环), 产品版本基线 v0.1.1; 本手册版本 v0.7.0, 覆盖上述全部产品版本;
- **容器镜像**: edgeflow/cloudcore:v0.7.0、edgeflow/edgecore:v0.7.0, 支持 linux/amd64 与 linux/arm64 双架构;
- **Helm Chart**: build/charts/edgeflow (Chart 包 edgeflow-0.7.0.tgz, version 0.7.0 / appVersion v0.7.0), 用于在 Kubernetes 集群中部署云端;
- **离线制品**: 发布目录 release/v0.7.0/, 包含云/边/管理工具(cloudcore/edgecore/keadm) 在 linux-amd64、linux-arm64、darwin-arm64 三个平台的 9 个二进制、校验和(checksums.txt)、软件物料清单 (SBOM) 与 Helm Chart 包, 共 12 个制品; 历史版本制品见 release/ 目录下对应版本子目录。

## 1.5 术语约定

| 术语                      | 说明                                                                                                                                                   |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| nodeID                  | 边缘节点唯一标识。edgecore 注册时上报，默认格式为 edge- <code>&lt; 主机名 &gt;</code> ，仅允许字母、数字、点、连字符与下划线（匹配 <code>^[A-Za-z0-9.-_]+\$</code> ，v0.4.0 起云端强制校验，含 / 的 ID 拒绝写入） |
| Pod                     | 云端下发的容器应用单元，包含名称（name）、命名空间（namespace）、镜像（image）与副本数（replicas）等描述                                                                                    |
| Mapper                  | 设备接入程序，负责协议解析、属性采集与指令执行（如 <code>mock_sensor</code> 、 <code>modbus</code> ）                                                                           |
| 数字孪生（Digital Twin）      | 设备的云端影子：desired 为云端下发的期望值，reported 为设备实际上报值                                                                                                          |
| 可靠下发（Reliable Delivery） | 下发类接口的消息投递机制：消息送达边缘并收到确认（Ack）后才返回成功，未确认则超时重试                                                                                                         |
| 调谐（Reconcile）           | 边缘侧周期性比对期望状态与实际状态，并驱动实际状态收敛到期望状态的过程                                                                                                                  |
| 嵌入式 etcd                | 云端单成员 etcd 实例（v0.4.0 起默认启用，监听 127.0.0.1:12379/12380，仅绑回环），作为云端的持久化后端，以写穿方式保存注册台账与设备期望状态                                                              |
| 外部 etcd 模式              | 设置 <code>EDGEFLOW_CLOUDCORE_ETCD_ENDPOINTS</code> 后，云端直连共享的 etcd 集群（v0.5.0 起；v0.6.0 起支持多副本真多活），不再内嵌 etcd 实例                                          |
| 写穿持久化（write-through）    | 写路径先写入 etcd 成功、再更新内存缓存：“写成功即已持久化”；读路径走内存缓存                                                                                                           |
| 模型仓库                    | 模型与版本两级台账（v0.7.0）：“镜像即模型、Tag 即版本”，登记镜像引用与 sha256 摘要，供发布与追踪                                                                                           |
| 灰度发布                    | 将模型版本按节点白名单或百分比分批下发到边缘（v0.7.0），支持批次节奏、fail-fast、取消与回滚                                                                                                |
| 部署影子                    | 模型发布后云端写穿的“版本—节点—时间”台账（v0.7.0），反映每台节点的期望部署版本                                                                                                         |
| 领跑锁                     | 外部多副本模式下灰度发布任务的租约锁（v0.7.0，TTL 默认 60s）：持有者崩溃后其余副本 $\leq$ TTL 内接管续跑                                                                                    |
| 命名空间（设备）                | 设备注册与指令路由的隔离维度（v0.2.0 起，如 Modbus 设备命名空间）：同名设备可分命名空间共存，指令按命名空间路由，错误命名空间被边缘拒绝                                                                          |

## 1.6 功能边界与已知限制

使用 EdgeFlow v0.7.0 前，请了解以下边界：

- **云端分级持久化 (v0.4.0 起)**: 节点注册台账与设备 Desired 跨重启保留; 心跳、Status、Pod 状态、设备 Properties 与 Offline 标记不落盘, 云端重启后短暂清空,  $\leq 1$  个上报周期自愈 (非永久丢失); 纯内存模式 (ETCD\_ENABLED=false 且未配置外部 etcd) 下模型发布任务与部署影子重启丢失 (L22);
- **接入令牌已消费**: k8s join 的 --token 写入 edgecore.env, edgecore 注册时随 Register 消息携带; 云端设置 EDGEFLOW\_CLOUDCORE\_NODE\_TOKEN 后启用校验, 不匹配的注册会被拒绝 (未设置时不校验, 向后兼容);
- **设备指令值类型**: device-command 的 value 为数值型 (float64), 字符串、布尔型属性暂不支持通过该端点下发;
- **Secret 明文存储**: config-sync 下发 Secret 时值为明文, 生产环境需自行加密;
- **无批量下发端点**: 当前版本未提供面向多节点的批量 (广播) 下发接口;
- **混合版本多副本禁止**: v0.5.0 与 v0.6.0 的 cloudcore 副本同连一个外部集群会导致旧版本误删活节点 (L15); v0.6.0 与 v0.7.0 混跑未验证 (L29) ——升级/回滚必须全停再全起;
- **OPC-UA 仅协议栈核心**: v0.3.0 起提供 UA Binary 协议栈 (编解码与 TCP 握手), SecurityPolicy None 明文传输, 仅限可信隔离网络, 严禁暴露公网; 设备读写服务与 Mapper 接入未实现;
- **nodeID 字符约束**: 仅允许字母、数字、点、连字符与下划线 (`^[A-Za-z0-9._-]+$`), 含 / 的节点 ID 拒绝写入 (v0.4.0 起)。

## 1.7 手册内容导航

本手册面向操作人员, 共分若干章节:

- 第 2 章 [chapter 二](#): 环境要求与安装部署——云端与边缘节点的安装、验证、卸载与升级回滚;
- 第 3 章 [chapter 三](#): 快速入门——单机跑通完整链路;
- 第 4 章 [chapter 四](#): 云端操作指南——REST API、监控指标与认证配置;
- 后续章节: 设备接入 (Mapper) 使用、边缘应用管理、维护与排障等专题。

## 第2章 环境要求与安装部署

本章内容：说明 EdgeFlow v0.7.0 的运行环境要求，并给出完整的安装部署步骤，包括获取软件、使用 `keadm` 初始化云端与接入边缘节点、Helm 部署，以及部署验证、卸载清理与升级回滚。

### 2.1 环境要求

| 部署角色            | 环境要求                                                                                                                       |
|-----------------|----------------------------------------------------------------------------------------------------------------------------|
| 云端 (cloudcore)  | 支持 linux/amd64 与 linux/arm64；建议 2 核 4 GB 以上；建议配置持久化存储：embed 模式使用数据目录（Helm 默认 PVC 1Gi, emptyDir 会丢数据），外部 etcd 模式需共享 etcd 集群 |
| Kubernetes 集群   | 使用 <code>keadm</code> 产物或 Helm 部署时需准备集群；需要 <code>kubect1</code> （与集群版本兼容）与 Helm v3+                                        |
| 边缘节点 (edgecore) | Linux 系统（支持 systemd）；Docker daemon 运行中（Edged 的容器运行时）；可访问云端 CloudHub 端口（默认 10000，NodePort 部署时为集群分配的节点端口）；SQLite 数据目录可写      |
| 管理机 (keadm)     | 任意 amd64/arm64 主机，仅用于生成离线部署产物，不直接操作集群                                                                                      |
| 源码构建            | Go 1.26+ 与 Make；构建容器镜像还需 Docker                                                                                            |
| 可选依赖            | mosquitto（MQTT 数据面）；缺失时主链路不受影响                                                                                             |
| 外部 etcd 集群（可选）  | 使用外部 etcd 模式（v0.5.0 起）时需准备：3 节点奇数、同地域、quota 256MiB、compaction 1h；生产建议启用 TLS/mTLS（非回环端点且无 TLS 时云端拒绝启动）                      |

### 2.2 获取软件

#### 2.2.1 方式一：源码构建

在仓库根目录执行：

```
make build
```

构建产物位于 `bin/` 目录：`bin/cloudcore`、`bin/edgecore`。`keadm` 可用 `go build -o bin/keadm ./cmd/keadm` 构建；`mock-cloudhub` 为联调工具，用 `go run ./hack/mock-cloudhub` 运行（无需构建）。

#### 2.2.2 方式二：使用发布制品

发布目录 `release/v0.7.0/` 提供预编译二进制：`cloudcore`、`edgecore`、`keadm` 每组件均含 `linux-amd64`、`linux-arm64` 与 `darwin-arm64` 三个平台版本（共 9 个二进制），以及

Helm Chart 包 `edgeflow-0.7.0.tgz`（历史版本制品见 `release/` 对应子目录）。下载后可使用 `checksums.txt` 校验文件完整性，并将二进制放入 `PATH` 或仓库 `bin/` 目录后按需执行。

### 2.2.3 方式三：容器镜像

可自行构建镜像，或直接使用已发布的 `edgeflow/cloudcore:v0.7.0`、`edgeflow/edgecore:v0.7.0`。本地构建命令（仓库根目录执行）：

```
docker build -f build/docker/Dockerfile --target cloudcore -t edgeflow/cloudcore:v0.7.0 .
docker build -f build/docker/Dockerfile --target edgecore -t edgeflow/edgecore:v0.7.0 .
```

## 2.3 安装云端（cloudcore）

### 2.3.1 方式一：keadm 生成 Kubernetes 部署产物（推荐）

`keadm init` 在管理机上生成云端部署产物，产物可提交到 Kubernetes 集群执行。参数如下：

| 参数                             | 默认值                                    | 说明                                                                                                  |
|--------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>--cloudcore-image</code> | <code>edgeflow/cloudcore:v0.7.0</code> | cloudcore 容器镜像                                                                                      |
| <code>--tls</code>             | 关                                      | 启用云边 mTLS，注入 TLS 相关环境变量                                                                             |
| <code>--tls-san</code>         | 空                                      | 证书 SAN (Subject Alternative Name)，逗号分隔，如 <code>IP:1.2.3.4,DNS:host</code> ；仅 <code>--tls</code> 时生效 |
| <code>--service-type</code>    | <code>NodePort</code>                  | Service 类型： <code>NodePort</code> （边缘跨集群接入）或 <code>ClusterIP</code> （仅集群内访问）                        |
| <code>--output-dir</code>      | <code>./keadm-out</code>               | 产物输出目录                                                                                              |

生成命令示例：

```
# 最简（明文通道，本地/测试用）
keadm init --output-dir=./keadm-out

# 生产推荐：启用云边 mTLS，并注入证书 SAN（边缘节点用 IP 接入时必须覆盖访问地址）
keadm init --tls --tls-san=IP:192.168.1.10 --output-dir=./keadm-out
```

产物说明：

- `cloudcore.yaml`：包含 Deployment（副本 1、探针指向 `/healthz`、安全上下文 `nonroot`）与 Service（`NodePort` 类型，暴露 HTTP 8080 与 CloudHub 10000 两个端口；hub 端口由集群自动分配在 30000-32767）；

- NOTES.txt: 部署步骤、验证方法、Helm 替代路径与 mTLS 说明。

在集群上执行产物:

```
kubectl apply -f cloudcore.yaml

# 验证
kubectl get deploy,svc,pods -l app.kubernetes.io/component=cloudcore
kubectl port-forward svc/edgeflow-cloudcore 8080:8080
curl http://127.0.0.1:8080/healthz          # 期望 HTTP 200

# 获取边缘节点接入用的 CloudHub 节点端口
kubectl get svc edgeflow-cloudcore -o jsonpath='{.spec.ports[?(@.name=="hub")].nodePort}'
```

### 2.3.2 方式二: Helm 部署

使用仓库内置 Chart 部署云端:

```
helm install edgeflow build/charts/edgeflow

# 集群外边缘节点接入时: 开启 hub 端口并改用 NodePort
helm install edgeflow build/charts/edgeflow \
  --set service.hubEnabled=true --set service.type=NodePort
```

验证部署:

```
kubectl get deploy,svc,pods -l app.kubernetes.io/instance=edgeflow
kubectl port-forward svc/edgeflow-cloudcore 8080:8080
curl http://127.0.0.1:8080/healthz
```

关键配置项: cloudcore.image.repository/tag(镜像地址与版本, 默认 edgeflow/cloudcore:v0.7.0)、service.type (默认 ClusterIP)、service.hubEnabled (默认 false, 集群外边缘接入需开启并配合 NodePort/LoadBalancer)。

**v0.4.0 起新增关键配置项** (云端持久化与高可用形态):

- cloudcore.replicaCount: 默认 1。embed 模式必须为 1——多副本各自内嵌 etcd 会脑裂, Chart 渲染期 {{ fail }} 守卫直接报错; 外部 etcd 模式 (v0.6.0 起) 可 >1, 此时自动注入 EDGEFLOW\_CLOUDCORE\_MULTI\_REPLICA=1 (真多活);
- cloudcore.etcd.enabled: 默认 true(注入 EDGEFLOW\_CLOUDCORE\_ETCD\_ENABLED=true)。设为 false 时强制纯内存模式并忽略 external.\* (数据重启丢失, 仅限测试);
- etcd.persistence.enabled/size: 默认 true/1Gi, PVC 挂载 /data。改为 emptyDir 后 etcd 数据不落盘, 持久化名存实亡, 生产环境请保持默认;
- etcd.external.enabled/endpoints: v0.5.0 起外部 etcd 模式——enabled 为 true 且 endpoints 非空时注入 EDGEFLOW\_CLOUDCORE\_ETCD\_ENDPOINTS (逗号连接), 跳过 embed、

不创建 PVC；endpoints 为空时渲染失败；

- `etcd.external.tls.{ca,cert,key}` 与 `etcd.external.allowInsecure`: v0.5.0 起——非回环端点且未配置 TLS 时云端拒绝启动(明文护栏), 仅限可信内网可用 `allowInsecure=true` 逃生；
- `etcd.external.nodeLeaseTTL`: v0.6.0——非空时注入 `EDGEFLOW_CLOUDCORE_NODE_LEASE_TTL` (默认 300s, 外部模式判活阈值)；
- `cloudcore.resources`: 建议 requests 256Mi / limits 1Gi (v0.4.0 起二进制体积与常驻内存上升)。

### 2.3.3 方式三：裸进程运行（单机联调）

开发或验证环境可直接运行二进制：

```
./bin/cloudcore
```

默认监听 REST API 8080 与 CloudHub 10000 端口；端口可通过配置文件(`config/cloudcore.json`, 可用 `--config` 指定路径)、环境变量或命令行覆盖, 优先级为命令行 > 环境变量 > 配置文件 > 默认值 (详见第 3 章 [chapter 三](#))。此方式仅用于单机联调, 生产环境请使用方式一或方式二。

## 2.4 接入边缘节点 (edgecore)

`keadm join` 在管理机或边缘节点上生成边缘接入产物。参数如下：

| 参数                            | 默认值                             | 说明                                                                                                                    |
|-------------------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>--cloudcore-ip</code>   | 必填                              | 云端 CloudHub 节点 IP (IPv4/IPv6 均可)                                                                                      |
| <code>--cloudcore-port</code> | 10000                           | CloudHub 端口；NodePort 部署时填集群分配的节点端口                                                                                    |
| <code>--token</code>          | 必填                              | 接入令牌：写入 <code>edgecore.env</code> 并由 <code>edgecore</code> 注册时携带；云端设 <code>EDGEFLOW_CLOUDCORE_NODE_TOKEN</code> 后启用校验 |
| <code>--node-id</code>        | <code>edge-&lt; 主机名 &gt;</code> | 边缘节点 ID, 仅允许字母、数字、点、连字符与下划线 ( <code>^[A-Za-z0-9._-]+\$</code> )                                                       |
| <code>--tls</code>            | 关                               | 启用 mTLS, 地址自动使用 <code>wss://</code>                                                                                   |
| <code>--output-dir</code>     | <code>./keadm-out</code>        | 产物输出目录                                                                                                                |

生成命令示例：

```
keadm join --cloudcore-ip=192.168.1.10 --token=<token> --output-dir=./keadm-out

# TLS 集群 + NodePort 部署 (端口为集群分配的 hub 节点端口)
keadm join --cloudcore-ip=192.168.1.10 --cloudcore-port=31000 \
  --token=<token> --node-id=edge-worker-01 --tls --output-dir=./keadm-out
```

产物说明:

- `edgecore.env`: 环境变量文件, 键名与 `edgecore` 读取的环境变量一一对应, 包括 `EDGEFLOW_EDGECORE_NODE`, `EDGEFLOW_EDGECORE_CLOUD_ADDR(ws://<ip>:<port>/v1/edge)`, `EDGEFLOW_EDGECORE_DB_PATH (/var/lib/edgeflow/edgeflow.db)`, `EDGEFLOW_EDGECORE_MQTT_ADDR(tcp://127.0.0.1:1883)` 等; `--tls` 时追加 TLS 与证书目录配置; `EDGEFLOW_EDGECORE_TOKEN` 为接入令牌键(`edgecore` 注册时携带, 云端设 `EDGEFLOW_CLOUDCORE_NODE_TOKEN` 后启用校验);
- `edgecore.service`: `systemd` 单元文件, 通过 `EnvironmentFile=/etc/edgeflow/edgecore.env` 加载配置, `ExecStart=/usr/local/bin/edgecore`, 崩溃自动重启;
- `install.sh`: 一键安装脚本 (需 `root`), 安装二进制、环境文件与 `systemd` 单元并启用服务;
- `README.md`: 接入说明与手动安装片段。

在边缘节点上执行:

```
# 1. 将产物与 edgecore 二进制 (与节点架构匹配) 拷贝到边缘节点
# 2. 执行安装脚本 (需 root)
sudo ./install.sh

# 3. 验证
systemctl status edgecore
journalctl -u edgecore -f
```

mTLS 说明: 启用 `--tls` 后, `edgecore` 首次运行会在 `/etc/edgeflow/certs` 自动生成或加载证书 (幂等); 云端侧需保证服务端证书 SAN 覆盖边缘节点访问的地址, 否则云边连接会被拒绝。

## 2.5 部署验证

部署完成后, 建议按以下清单验证:

- 云端 `/healthz` 返回 HTTP 200 与版本信息;
- 边缘节点已注册: 云端查询 `/api/v1/nodes` 能看到节点且状态为 `Ready` (查询方法见第 4 章 [chapter 四](#));
- `systemctl status edgecore` 显示 `active (running)`, 日志无致命错误;
- 可下发一个测试 Pod 验证端到端链路 (见第 3 章 [chapter 三](#));
- 配置热重载: `cloudcore` 与 `edgecore` 支持 `SIGHUP` 触发与每 60 秒轮询两种热重载方式 (云端 HTTP 端口可热切换, 部分配置项变更需重启, 生效边界见第 5 章 [chapter 五](#))。



## 2.6 卸载与清理

```
# 清理 keadm 生成的产物（交互确认；--force 跳过确认，幂等）
keadm reset --output-dir=./keadm-out

# 卸载 Helm 部署
helm uninstall edgeflow

# 清理 Edged 管理的容器（按标签精确删除，不影响其他容器）
docker rm -f $(docker ps -aq --filter label=edgeflow.pod) 2>/dev/null

# 清理本地 SQLite 元数据（生产环境请按备份策略处理）
rm -f data/edgeflow.db
```

## 2.7 升级与回滚

keadm 在产物层面提供升级与回滚能力（执行前自动备份并记录操作台账）：

```
# 升级产物到新版本（操作人经环境变量传入，写入操作台账）
export KEADM_OPERATOR=alice
keadm upgrade --version=v0.7.0 --output-dir=./keadm-out

# 回滚到最近一次备份
keadm rollback --latest --output-dir=./keadm-out

# 查询操作台账
keadm ops-ledger --limit=10
```

升级或回滚后需重新应用产物（`kubectl apply` 或 `./install.sh`）。注意：v0.4.0 起云端为分级持久化——注册台账与设备 Desired 跨重启保留，重启后节点短暂 Unknown，边缘节点下一次心跳（默认 30s）即翻新为 Ready；Pod 状态与上报数据重启后 ≤1 个上报周期自愈。云端多副本（外部 etcd 模式）升级/回滚务必全停再全起，禁止混合版本混跑（详见第 7 章 [chapter 七](#)）。

## 2.8 常见问题

| 现象                                                        | 处理                                                                                                                                                                               |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 缺少必填参数 <code>--cloudcore-ip</code> 或 <code>--token</code> | 补全对应参数后重试                                                                                                                                                                        |
| <code>--node-id</code> 含空白字符                              | 使用 <code>--node-id=edge-xxx</code> 显式指定合法 ID                                                                                                                                     |
| <code>kubectl apply</code> 报 schema 校验失败                  | 检查 <code>kubectl</code> 版本与集群 API 版本是否兼容                                                                                                                                         |
| edgecore 起不来                                              | <code>journalctl -u edgecore -e</code> 查看日志；确认可达 <code>--cloudcore-ip</code> 的 hub 端口；确认 <code>env</code> 文件键值未被手改                                                               |
| 云边连接被拒 (TLS)                                              | 证书 SAN 未覆盖访问地址，云端以 <code>--tls-san=IP:&lt;访问 IP&gt;</code> 重新 <code>init</code> 并 <code>apply</code>                                                                             |
| Pod 容器未创建                                                 | 确认 Docker daemon 运行 ( <code>docker info</code> )；镜像本地未缓存时首次拉取需要时间                                                                                                                |
| 云端重启后节点仍可见但状态 Unknown                                     | v0.4.0 起注册台账跨重启保留，属正常现象：等待边缘节点下一次心跳（默认 30s）后自动翻新为 Ready，无需重新注册；Pod 状态与上报数据同样 $\leq 1$ 个上报周期自愈                                                                                    |
| embed 模式部署多副本失败（渲染报错）                                     | Chart 内置 <code>{{ fail }}</code> 守卫：embed 模式 <code>replicaCount</code> 必须为 1（多副本各自内嵌 etcd 会脑裂）；需要多副本请改用外部 etcd 模式（v0.6.0 起支持）                                                    |
| 外部 etcd 模式启动失败                                            | 按启动日志区分三类原因：探活失败（Unavailable，地址/连通性错误）、鉴权被拒（PermissionDenied，证书或 etcd 权限角色问题）、明文护栏（非回环端点且未配置 TLS 被拒绝启动，需配置 TLS 或仅在可信内网用 <code>ETCD_ALLOW_INSECURE=1</code> 逃生）；详细排障见部署文档 §10.7.6 |

## 第3章 快速入门

本章内容：在单台开发机上用约 10 分钟跑通 EdgeFlow 完整链路：构建制品、启动云端与边缘、注册节点、下发 Pod、查看容器、查看设备数据、下发设备指令，并可选体验模型发布。适合第一次接触 EdgeFlow 的操作人员。

### 3.1 准备工作

开始前请确认：

- 本机为 macOS 或 Linux；
- Docker 已安装且 daemon 运行中（Edged 的容器运行时）；
- 已安装 curl；源码构建还需 Go 1.26+ 与 Make；
- 已进入 EdgeFlow 仓库根目录（`cd edgeflow`）；
- 可选：mosquitto（MQTT 数据面），缺失时跳过不影响主链路。

```
docker info      # 确认 Docker daemon 运行
go version       # 确认 Go 版本（源码构建时需要）
```

### 3.2 第一步：构建制品

```
make build
```

构建产物位于 `bin/:bin/cloudcore,bin/edgecore`。也可以直接使用发布目录 `release/v0.7.0/` 中的预编译二进制（获取方式见第 2 章 [chapter 二](#)）。

### 3.3 第二步：启动云端 cloudcore

打开终端 A，执行：

```
./bin/cloudcore
```

预期日志输出 `HTTP server listening on :8080`。默认监听：REST API 端口 8080、CloudHub WebSocket 端口 10000。端口可通过环境变量覆盖（`EDGEFLOW_CLOUDCORE_PORT`、`EDGEFLOW_CLOUDCORE_HUB_PORT`），命令行参数 `--port` 优先级更高。

验证健康检查：

```
curl http://127.0.0.1:8080/healthz
```

预期返回 HTTP 200 与 `{"status":"ok","version":{...}}`。

### 3.4 第三步：启动边缘 edgecore

打开终端 B，执行：

```
EDGEFLOW_EDGECORE_NODE_ID=node-1 \  
EDGEFLOW_EDGECORE_CLOUD_ADDR=ws://127.0.0.1:10000/v1/edge \  
./bin/edgecore
```

说明：

- `EDGEFLOW_EDGECORE_NODE_ID`：节点 ID。不设置时默认为 `edge-< 主机名 >`，仅允许字母、数字、点、连字符与下划线；
- `EDGEFLOW_EDGECORE_CLOUD_ADDR`：云端 CloudHub 地址（明文通道用 `ws://`；启用 mTLS 时用 `wss://`，见第 2 章 [chapter 二](#)）；
- `edgecore` 启动后自动完成节点注册、云边连接，并随进程启动内置模拟温湿度传感器 `sensor-01` (`mock_sensor`)，无需额外配置。

### 3.5 第四步：验证节点注册

回到终端 A，查询节点列表：

```
curl http://127.0.0.1:8080/api/v1/nodes
```

预期返回 JSON 数组，包含 `nodeID` 为 `node-1`、`status` 为 `Ready` 的节点。若长时间未出现，请检查终端 B 的 `edgecore` 日志与网络连通性。

### 3.6 第五步：下发第一个 Pod (nginx)

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/node-1/podsync \  
-H 'Content-Type: application/json' \  
-d '{"operation":"add","pod":{"name":"nginx","namespace":"default","image":"nginx:1.25-alpine","replicas":1}}'
```

预期返回 `{"status":"ok","acked":true}`，表示边缘节点已确认。镜像首次拉取可能需要一些时间。

如需限制容器资源（可选，v0.2.0 起支持），在 `pod` 中追加 `resources` 字段（K8s 风格，零值表示不限制）：

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/node-1/podsync \  
-H 'Content-Type: application/json' \  
-d '{"operation":"add","pod":{"name":"nginx","namespace":"default","image":"nginx:1.25-alpine","replicas":1,"resources":{"cpuRequest":"100m","cpuLimit":"250m","memoryRequest":"64Mi","memoryLimit":"128Mi"}}}'
```

资源语义 (v0.2.0): 请求量大于上限 (`request > limit`) 返回 400; 超出节点超卖容量返回 409 (`EDGEFLOW_RESOURCE_EXHAUSTED`, 不落盘不建容器); 格式非法 (如 NaN/Inf/前导 +) 同样返回 400。

### 3.7 第六步: 查看容器与 Pod 状态

在边缘侧查看容器 (本机即边缘):

```
docker ps --filter label=edgeflow.pod
```

预期看到容器 `edgeflow-default-nginx-0` 处于运行状态。

查询云端 Pod 状态:

```
curl http://127.0.0.1:8080/api/v1/pods
```

预期返回 `phase` 为 `Running` 的 Pod 记录。

注: 资源漂移说明 (v0.2.0 起): 在边缘侧直接修改容器资源限制 (如 `docker update`) 会被 Edged 判定为与期望状态不一致并自动重建容器 (每轮最多重建 1 个), 资源以云端下发为准。

### 3.8 第七步: 查看设备数据

```
curl http://127.0.0.1:8080/api/v1/devices
```

预期返回设备 `sensor-01`, `properties` 中包含 `temperature` 与 `humidity` 两个属性 (周期上报的实时值)。设备默认上报周期为 30 秒, 可设置 `EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL=3s` 加速观察。

### 3.9 第八步: 下发设备指令

向模拟传感器下发目标温度指令:

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/node-1/device-command \
-H 'Content-Type: application/json' \
-d '{"deviceName":"sensor-01","property":"targetTemp","value":25}'
```

预期返回 `{"status":"ok","acked":true}`。再次查询设备:

```
curl http://127.0.0.1:8080/api/v1/devices
```

预期 `sensor-01` 的 `desired` 中写入 `{"targetTemp":25}`, 表明指令已被边缘执行并写回云端设备状态。

### 3.10 体验模型发布 (可选, v0.7.0)

以下步骤在已完成第八步的同一环境上体验模型仓库与灰度发布。该能力为云端内置 (v0.7.0), 边缘零代码改动; 发布者复用既有 `podsync` 与 `config-sync` 经可靠投递下发, 旧版 `edgecore` 直

接可用。前置条件：`node-1` 处于 `Ready` 状态（百分比发布的分母为创建时刻 `Ready` 节点数，0 台会返回 422）。

### 3.10.1 第一步：注册模型

```
curl -X POST http://127.0.0.1:8080/api/v1/models \
  -H 'Content-Type: application/json' \
  -d '{"name":"defect-detector","description":"缺陷检测模型","type":"detection"}'
```

预期返回 200 与完整 Model 对象（含 `createdAt/updatedAt`）。模型名全局唯一（重复创建返回 409）。

### 3.10.2 第二步：登记版本

```
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/versions \
  -H 'Content-Type: application/json' \
  -d '{"version":"v1.2.0","mirror":"registry.example.com/edgeflow/models/defect-detector:v1.2.0","sha256":"sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08","archs":["amd64","arm64"],"metadata":{"threshold":"0.8"}}'
```

预期返回 200 与 `ModelVersion`（`status` 为 `draft`）。“Tag 即版本”：`mirror` 必须带 tag，`sha256` 为镜像防篡改登记（`^sha256:[0-9a-f]{64}$`）。

### 3.10.3 第三步：激活版本

```
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/versions/v1.2.0/activate
```

预期返回 200；版本状态 `draft` → `active`（自动降级旧 `active` 版本）。发布目标必须是 `active` 版本。

### 3.10.4 第四步：全量发布

```
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/releases \
  -H 'Content-Type: application/json' \
  -d '{"version":"v1.2.0","target":{"type":"percentage","percentage":100}}'
```

预期返回 202 与 `release` 对象（`status` 为 `pending`，`targetNodes` 已物化）。发布为异步受理：可用 `GET /api/v1/models/defect-detector/releases/<releaseID>` 跟踪执行进度，全部目标节点 `deployed` 后进入 `succeeded`。

### 3.10.5 第五步：查询部署影子

```
curl http://127.0.0.1:8080/api/v1/models/defect-detector/deployments
```

预期返回每台目标节点的“版本—节点—时间”记录(model/version/mirror/releaseID/updatedAt), 即云端写穿的部署影子。

说明: 模型发布为云端能力——发布器自动执行 podsync (Pod 名 `edgeflow-model-< 模型名 >`、命名空间固定 `edgeflow`、`replicas=1`) 与 config-sync (ConfigMap 携带 `model/version/mirror/sha256/type/releasedAt` 与版本参数), 两步均被边缘确认后写穿部署影子。注意: **发布成功 镜像可用**——拉取发生在边缘, 请以试点节点 PodStatus (Running/CrashLoop) 为准; 灰度节奏建议先白名单 1 台试点 → 小批 (`batchSize+pauseBetween`) → 全量; 生产环境建议 `embed/外部 etcd` 模式 (纯内存模式下发布任务与部署影子重启丢失)。

### 3.11 一键 Demo (可选)

仓库提供端到端演示脚本, 自动完成上述全部步骤并输出 DEMO PASS:

```
bash examples/demo.sh
```

脚本自动完成: 构建、挑选空闲端口、启动 `cloudcore` 与 `edgecore` (运行时数据落在临时目录)、验证节点注册、下发 `nginx` Pod、验证容器与 Pod 状态、验证设备数据流、下发设备指令、可选 MQTT 数据面验证, 最后自动清理资源。脚本幂等, 可重复运行; 设置 `EDGEFLOW_DEMO_SKIP_BUILD=1` 可跳过构建步骤。

### 3.12 清理

```
# 1. 回收 Pod (边缘 Edged 会自动删除容器)
curl -X POST http://127.0.0.1:8080/api/v1/nodes/node-1/podsync \
  -H 'Content-Type: application/json' \
  -d '{"operation": "delete", "pod": {"name": "nginx", "namespace": "default"}}'

# 2. 停止 cloudcore 与 edgecore (终端 A/B 按 Ctrl+C, 优雅退出)

# 3. 兜底清理容器与本地元数据 (可选)
docker rm -f $(docker ps -aq --filter label=edgeflow.pod) 2>/dev/null
rm -f data/edgeflow.db
```

3.13 常见问题

| 现象                     | 处理                                                                             |
|------------------------|--------------------------------------------------------------------------------|
| 节点未注册或状态非 Ready        | 查看 edgecore 日志; 确认 CloudHub 端口 (默认 10000) 可达、云边地址配置正确                          |
| podsync 返回 504 或长时间未确认 | 边缘未确认下发: 确认 edgecore 正在运行, 且下发路径中的节点 ID 与注册的 nodeID 完全一致                       |
| 容器未创建                  | 确认 Docker daemon 运行 ( <code>docker info</code> ); 镜像首次拉取需要时间                   |
| 设备无数据上报                | 上报周期默认 30 秒, 可设置 <code>EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL=3s</code> 加速观察 |



## 第4章 云端操作指南

本章内容：介绍 cloudcore 对外提供的全部 REST API 端点——用途、请求示例与响应字段说明，以及监控指标 (/metrics)、OCSP 在线吊销查询 (/ocsp) 与 API 认证的启用方法。v0.7.0 起总端点数为 31（14 个既有端点 + 17 个模型 API 端点，详见 §4.2 与 §4.11）。操作人员可通过 curl 或任意 HTTP 客户端调用。

### 4.1 通用约定

- **Base URL**: `http://<cloudcore-ip>:8080` (端口可用 `--port` 或环境变量 `EDGEFLOW_CLOUDCORE_PORT` 覆盖)；
- **数据格式**: JSON；请求需携带 `Content-Type: application/json`，响应同样为 JSON；
- **时间戳**: Unix 毫秒（注册、心跳、上报时间等）；
- **List 风格**: `edgenodes`、`pods`、`devices` 端点采用 Kubernetes List 风格 (`kind/apiVersion/items`)，空数据编码为 `[]` 而非 `null`；`/api/v1/nodes` 为裸数组；
- **路径参数**: `{nodeID}` 为边缘节点 ID (edgecore 注册时上报，默认 `edge-<主机名>`)，必须与注册时的 `nodeID` 完全一致；`nodeID` 须匹配 `^[A-Za-z0-9._-]+$` (v0.4.0 起，含 / 的 `nodeID` 拒绝写入)；
- **状态语义 (分级持久化, v0.4.0 起)**: 云端三块存储 (`registry/podstatus/devicestatus`) 由纯内存改为写穿 (write-through) 持久化——节点注册台账与设备 `Desired` 跨重启保留；心跳、`Status`、`Pod` 状态、设备上报属性与 `Offline` 标记不落盘，重启后短暂清空 (≤1 上报周期自愈，非永久丢失)；边缘侧元数据由 SQLite 持久化，不受影响；
- **并发语义 (v0.6.0 多副本)**: 外部 etcd 多副本形态下，设备 `Desired` 写入为 CAS (Compare-And-Swap: 先读基准 revision，无变化才写入)；并发冲突重试耗尽时 HTTP 仍返回 200，仅日志输出 `concurrent-write`——属正常语义而非错误，重发指令即可收敛；
- **OpenAPI 契约**: OpenAPI v3 契约见 `docs/openapi/edgeflow-openapi.yaml` (由 `hack/gen-openapi` 重新生成，请勿手工编辑)；API 兼容性契约测试见 `tests/contract` (v0.7.0 起覆盖全部 31 个端点，含 17 个模型 API 端点)。

完整端点总览见下一节。

### 4.2 端点总览

总 HTTP 端点 14→31 (v0.7.0 新增 17 个模型 API 端点)；既有 14 个端点行为逐字节不变 (回归锚点)。以下为既有端点 (14 个)：

| 方法   | 路径                                   | 说明                                                                  | 主要状态码                             |
|------|--------------------------------------|---------------------------------------------------------------------|-----------------------------------|
| GET  | /healthz                             | 健康检查（探针用）                                                           | 200                               |
| GET  | /metrics                             | Prometheus 指标（五指标）                                                  | 200                               |
| GET  | /api/v1/nodes                        | 全部节点（运行视角）                                                          | 200                               |
| GET  | /api/v1/nodes/{nodeID}               | 单节点详情                                                               | 200 / 404                         |
| GET  | /api/v1/edgenodes                    | 全部节点（CRD 对象视角）                                                      | 200                               |
| GET  | /api/v1/edgenodes/{nodeID}           | 单节点 EdgeNode 对象                                                     | 200 / 404                         |
| GET  | /api/v1/pods                         | 全部节点 Pod 状态                                                         | 200                               |
| GET  | /api/v1/nodes/{nodeID}/pods          | 单节点 Pod 状态                                                          | 200 / 404                         |
| GET  | /api/v1/devices                      | 全部设备状态                                                              | 200                               |
| GET  | /api/v1/nodes/{nodeID}/devices       | 单节点设备状态                                                             | 200 / 404                         |
| POST | /api/v1/nodes/{nodeID}/pods          | 下发 Pod 配置（add/update/delete, 含资源诉求）                                 | 200 / 400 / 404 / 409 / 502 / 504 |
| POST | /api/v1/nodes/{nodeID}/configs       | 下发 ConfigMap/Secret 配置                                              | 200 / 400 / 404 / 502 / 504       |
| POST | /api/v1/nodes/{nodeID}/devicecommand | 下发设备指令（期望值）                                                         | 200 / 400 / 404 / 502 / 504       |
| POST | /ocsp                                | OCSP 在线吊销查询（RFC 6960, DER 编码; per-IP 限流默认 10 req/s + Cache-Control) | 200 / 400 / 429 / 500             |

v0.7.0 新增模型 API（17 个，全部 /api/v1/\*，详见 §4.11）：

| 方法     | 路径                                                       | 说明                                                                                       | 主要状态码                       |
|--------|----------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------|
| GET    | /api/v1/models                                           | 模型列表 (K8s 风格, 按 name 排序; v0.8.0 起支持 limit(1-1000)/offset(0) 分页, 缺省全量, 响应头 X-Total-Count) | 200 / 400                   |
| POST   | /api/v1/models                                           | 创建模型                                                                                     | 200 / 400 / 409             |
| GET    | /api/v1/models/{modelName}                               | 模型详情                                                                                     | 200 / 404                   |
| PUT    | /api/v1/models/{modelName}                               | 更新模型 (description/-type/metadata)                                                        | 200 / 400 / 404 / 409       |
| DELETE | /api/v1/models/{modelName}                               | 删除模型 (无 active 版本、无在途发布; 级联 draft/archived 版本 + 部署影子)                                    | 200 / 404 / 409             |
| GET    | /api/v1/models/{modelName}/versions                      | 版本列表 (按 tag 排序); 模型不存在 → 404                                                             | 200 / 404                   |
| POST   | /api/v1/models/{modelName}/versions                      | 创建版本 (初始 draft)                                                                          | 200 / 400 / 404 / 409       |
| GET    | /api/v1/models/{modelName}/versions/{version}            | 版本详情                                                                                     | 200 / 404                   |
| DELETE | /api/v1/models/{modelName}/versions/{version}            | 删除版本 (仅 draft/archived)                                                                  | 200 / 404 / 409             |
| POST   | /api/v1/models/{modelName}/versions/{version}/activate   | 激活版本 (仅 draft/archived 降级旧 active)                                                       | 200 / 400 / 404 / 409       |
| POST   | /api/v1/models/{modelName}/versions/{version}/archive    | 归档版本 (仅 active)                                                                          | 200 / 404 / 409             |
| POST   | /api/v1/models/{modelName}/releases                      | 创建发布 (异步执行)                                                                              | 202 / 400 / 404 / 409 / 422 |
| GET    | /api/v1/models/{modelName}/releases                      | 发布列表 (按 createdAt 升序)                                                                    | 200 / 404                   |
| GET    | /api/v1/models/{modelName}/releases/{releaseID}          | 发布详情 (含 releaseID 汇总)                                                                    | 200 / 404                   |
| POST   | /api/v1/models/{modelName}/releases/{releaseID}/cancel   | 取消发布 (异步执行)                                                                              | 200 / 404 / 409             |
| POST   | /api/v1/models/{modelName}/releases/{releaseID}/rollback | 回滚发布 (异步执行, 逆序批量)                                                                        | 202 / 400 / 404 / 409 / 422 |
| GET    | /api/v1/models/{modelName}/deployments                   | 部署影子 (版本一节点一时间追踪)                                                                        | 200 / 404                   |

注：31 = 14 既有 (含 /healthz、/metrics、/ocsp 三个非 /api/v1/\* 端点) + 17 模型 API; 全部模型 API 注册于既有 apiMux, EDGEFLOW\_CLOUDCORE\_AUTH=on 时自动要求 Authorization: Bearer <token> (401 自动生效), 审计台账自动记录。

## 4.3 健康检查

### 4.3.1 GET /healthz

用途：服务探活，返回云端运行状态与版本信息（Helm 部署的存活/就绪探针均指向此路径）。

```
curl http://127.0.0.1:8080/healthz
```

响应示例（HTTP 200）：

```
{"status": "ok", "version": {"version": "v0.7.0", "gitCommit": "c7aeec1", "buildTime": "2026-08-25T18:00:00+0800", "goVersion": "go1.26.2"}}
```

多副本语义（v0.6.0）：外部 etcd 模式且 EDGEFLOW\_CLOUDCORE\_MULTI\_REPLICA=1 时，/healthz 反映 etcd 连接状态——etcd 失联超过节点租约 TTL(EDGEFLOW\_CLOUDCORE\_NODE\_LEASE\_TTL, 默认 300s) 返回 503, K8s liveness 探针据此重启副本自愈；其余形态（纯内存、embed、单副本外部模式）恒返回 200（进程存活语义，不反映存储状态）。

## 4.4 节点 API

### 4.4.1 GET /api/v1/nodes ——全部节点（运行视角）

用途：获取全部已注册节点，直接返回节点元数据数组（按 nodeID 排序），用于快速了解节点在线状态与平台信息。

```
curl http://127.0.0.1:8080/api/v1/nodes
```

响应示例（HTTP 200，无节点时返回 []）：

```
[
  {
    "nodeID": "edge-node-1",
    "nodeName": "edge-node-1",
    "arch": "arm64",
    "os": "darwin",
    "edgecoreVersion": "version=v0.7.0 gitCommit=c7aeec1 goVersion=go1.26.2",
    "cpu": 8,
    "memory": 0,
    "ip": "127.0.0.1",
    "registeredAt": 1786705914423,
    "lastHeartbeatAt": 1786705914423,
    "status": "Ready"
  }
]
```

字段说明：

| 字段                             | 类型           | 说明                                             |
|--------------------------------|--------------|------------------------------------------------|
| nodeID                         | string       | 节点唯一 ID (edgecore 的 EDGEFLOW_EDGECORE_NODE_ID) |
| nodeName                       | string       | 云端分配的节点名 (当前与 nodeID 一致)                       |
| arch / os                      | string       | edgecore 所在平台 (注册时上报)                          |
| edgecoreVersion                | string       | edgecore 版本串                                   |
| cpu / memory                   | int / uint64 | 节点资源 (内存: Linux 上报实际值, 非 Linux 平台为 0)          |
| ip                             | string       | 连接来源 IP                                        |
| registeredAt / lastHeartbeatAt | int64        | 注册时间 / 最近心跳时间 (Unix 毫秒)                        |
| status                         | string       | Ready / Unknown / Offline                      |

#### 4.4.2 GET /api/v1/nodes/{nodeID} ——单节点详情

用途: 查询指定节点。节点不存在时返回 404 与 {"error": "node not found", ...}。

```
curl http://127.0.0.1:8080/api/v1/nodes/edge-node-1
```

#### 4.4.3 GET /api/v1/edgenodes ——全部节点 (CRD 视角)

用途: 以 EdgeNode 对象 (apiVersion: edgeflow.io/v1alpha1) 形式获取节点, Kubernetes List 风格, 便于按 CRD 对象消费。

```
curl http://127.0.0.1:8080/api/v1/edgenodes
```

响应示例 (HTTP 200):

```
{
  "kind": "EdgeNodeList",
  "apiVersion": "edgeflow.io/v1alpha1",
  "items": [
    {
      "apiVersion": "edgeflow.io/v1alpha1",
      "kind": "EdgeNode",
      "metadata": {"name": "edge-node-1", "uid": "...", "creationTimestamp": "2026-08-14T19:11:54+08:00"},
      "spec": {"nodeID": "edge-node-1", "role": "edge"},
      "status": {"phase": "Running", "heartbeatTime": "...", "conditions": [{"type": "Ready", "status": "True"}]}
    }
  ]
}
```

关键字段: `spec.nodeID` (节点唯一标识)、`spec.role` (角色, 默认 `edge`)、`status.phase` (`Pending/Running/Offline/Unknown`)、`status.conditions` (健康条件, 如 `Ready`)。

#### 4.4.4 GET /api/v1/edgenodes/{nodeID} ——单节点 EdgeNode 对象

用途: 查询指定节点的 EdgeNode 对象; 节点不存在时返回 404。

```
curl http://127.0.0.1:8080/api/v1/edgenodes/edge-node-1
```

### 4.5 Pod API

#### 4.5.1 GET /api/v1/pods ——全部 Pod 状态

用途: 获取云端保存的全部 Pod 状态 (由边缘上报)。

```
curl http://127.0.0.1:8080/api/v1/pods
```

响应示例 (HTTP 200):

```
{
  "kind": "PodStatusList",
  "apiVersion": "v1",
  "items": [
    {
      "nodeID": "edge-node-1",
      "podName": "nginx",
      "namespace": "default",
      "phase": "Running",
      "message": "",
      "lastReconcileAt": 1786705732883
    }
  ]
}
```

字段说明: `phase` 取 `Running / Stopped / Absent / Error / Unknown`; `message` 为附加说明 (如错误原因); `lastReconcileAt` 为边缘最近一次调谐时间 (Unix 毫秒)。

#### 4.5.2 GET /api/v1/nodes/{nodeID}/pods ——单节点 Pod 状态

用途: 查询指定节点的 Pod 状态。语义约定: 节点不存在 (从未注册) 返回 404; 节点存在但无 Pod 返回 200 与空 `items`。

```
curl http://127.0.0.1:8080/api/v1/nodes/edge-node-1/pods
```

## 4.6 设备 API

### 4.6.1 GET /api/v1/devices ——全部设备状态

用途：获取全部设备的数字孪生状态：properties 为设备实际上报值，desired 为云端下发的期望值（设备上报不会覆盖 desired）。

```
curl http://127.0.0.1:8080/api/v1/devices
```

响应示例（HTTP 200，内置模拟传感器 sensor-01）：

```
{
  "kind": "DeviceStatusList",
  "apiVersion": "v1",
  "items": [
    {
      "nodeID": "edge-node-1",
      "deviceName": "sensor-01",
      "namespace": "default",
      "properties": {"humidity": 46.39, "temperature": 29.38},
      "desired": {"targetTemp": 25},
      "lastReportedAt": 1786705917423
    }
  ]
}
```

字段说明：properties / desired 均为 map[string]float64；lastReportedAt 为最近上报时间（Unix 毫秒）。

### 4.6.2 GET /api/v1/nodes/{nodeID}/devices ——单节点设备状态

用途：查询指定节点的设备状态。语义与单节点 Pod 查询一致：节点不存在返回 404；存在但无设备返回 200 与空 items。

```
curl http://127.0.0.1:8080/api/v1/nodes/edge-node-1/devices
```

## 4.7 下发类 API（可靠下发）

三个下发端点共用同一套可靠投递语义：消息进入 CloudHub 发送缓冲，由边缘 EdgeHub 接收并处理，处理完成后回确认（Ack）给云端，云端收到确认后返回 200；未确认时按 5 秒超时重试，最多尝试 3 次。

### 4.7.1 POST /api/v1/nodes/{nodeID}/podsync ——下发 Pod 配置

用途：向边缘节点下发 Pod 期望配置（新增/更新/删除），由边缘 Edged 调谐容器。

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/podsync \
-H 'Content-Type: application/json' \
```

```
-d '{"operation":"add","pod":{"name":"nginx","namespace":"default","image":"nginx":1.25-alpine","replicas":1,"resources":{"cpuRequest":"100m","cpuLimit":"250m","memoryRequest":"64Mi","memoryLimit":"128Mi"}}}'
```

请求体字段:

| 字段                          | 类型     | 必填            | 说明                                   |
|-----------------------------|--------|---------------|--------------------------------------|
| operation                   | string | 是             | add / update / delete (白名单校验)        |
| pod.name                    | string | 是             | Pod 名称(delete 时作为删除键之一)              |
| pod.namespace               | string | 否             | 命名空间(缺省 default)                     |
| pod.image                   | string | add/update 必填 | 容器镜像(delete 不需要)                     |
| pod.replicas                | int    | 否             | 副本数(Edged 按此保证多副本)                   |
| pod.resources.cpuRequest    | string | 否             | CPU 请求量(K8s 风格, 如 100m; 零值/缺省 = 不限制) |
| pod.resources.cpuLimit      | string | 否             | CPU 上限(如 250m; 零值/缺省 = 不限制)          |
| pod.resources.memoryRequest | string | 否             | 内存请求量(如 64Mi; 零值/缺省 = 不限制)           |
| pod.resources.memoryLimit   | string | 否             | 内存上限(如 128Mi; 零值/缺省 = 不限制)           |

响应示例 (HTTP 200, 边缘已确认):

```
{"status":"ok","acked":true}
```

操作说明: 边缘收到后, 容器按 `edgeflow-< 命名空间 >-< 名称 >-< 序号 >` 命名并打标签; `delete` 时按命名空间与名称删除元数据, Edged 回收对应容器。

## 资源语义 (v0.2.0)

`pod.resources` 四个字段为 v0.2.0 新增 (云边契约只增不改, `omitempty`; 旧版边缘未配置资源时行为不变):

- **云端前置校验:** 数值格式非法 (含 NaN/Inf/溢出/前导 +) 或 `request > limit` → 400, 文案含超标字段 (如 CPU request (500m) 不能超过 CPU limit (250m));
- **超卖准入:** 节点已部署 request 求和 + 新请求超出节点容量 × 超卖率 → 409 `EDGEFLOW_RESOURCE_EXHAUST` (云端拒绝, 不落盘、不建容器; 重试无意义, 见下文八态响应语义);
- **边缘准入:** `admitPodResources` fail-closed——边缘侧同样校验, 不满足时以 `error Ack` 返回 502;
- **容器落地:** Docker `--cpus / --memory` (`--memory-swap` 禁用, 即内存不可换出);



- **资源漂移检测**：外部通过 `docker update` 等修改容器资源限制 → Edged 检测到与声明不符后自动 `stop` + 重建容器（镜像漂移同理），每轮最多重建 1 个（滚动门控）；漂移重建不计入 `RestartCount`；
- **节点容量**：默认自动探测，可用环境变量覆盖（见下表）。

资源调度环境变量（v0.2.0, edgcore 侧）：

| 环境变量                                     | 默认    | 说明                                |
|------------------------------------------|-------|-----------------------------------|
| EDGEFLOW_EDGECORE_NODE_CPU_MILLI         | 自动探测值 | 节点 CPU 容量（毫核）；显式设置覆盖探测值           |
| EDGEFLOW_EDGECORE_NODE_MEMORY_BYTES      | 自动探测值 | 节点内存容量（字节）；显式设置覆盖探测值（仅 Linux 可探测） |
| EDGEFLOW_EDGECORE_OVERCOMMIT_CPU_RATE    | 1.5   | CPU 超卖率：准入容量 = 节点容量 × 超卖率         |
| EDGEFLOW_EDGECORE_OVERCOMMIT_MEMORY_RATE | 1.5   | 内存超卖率：准入容量 = 节点容量 × 超卖率           |

#### 4.7.2 POST /api/v1/nodes/{nodeID}/config-sync —— 下发配置

用途：向边缘节点下发 ConfigMap/Secret 配置，供边缘应用使用。

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/config-sync \
-H 'Content-Type: application/json' \
-d '{"operation": "add", "config": {"name": "app-config", "namespace": "default", "kind": "ConfigMap", "data": {"key1": "value1"}}}'
```

请求体字段：

| 字段               | 类型                | 必填            | 说明                                   |
|------------------|-------------------|---------------|--------------------------------------|
| operation        | string            | 是             | add / update / delete                |
| config.name      | string            | 是             | 配置名称                                 |
| config.namespace | string            | 否             | 命名空间（缺省 default）                     |
| config.kind      | string            | 是             | ConfigMap / Secret（白名单校验）            |
| config.data      | map[string]string | add/update 必填 | 键值数据（Secret 的 value 当前为明文存储，生产需自行加密） |

响应示例（HTTP 200）：{"status": "ok", "acked": true}。

#### 4.7.3 POST /api/v1/nodes/{nodeID}/device-command —— 下发设备指令

用途：向边缘设备下发期望值指令，由对应 Mapper 执行；执行结果写入边缘设备孪生并随周期上报回云端。内置 `mock_sensor` 支持 `targetTemp` 与 `reset` 指令。

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/device-command \
-H 'Content-Type: application/json' \
-d '{"deviceName":"sensor-01","namespace":"default","property":"targetTemp","value":25}'
```

请求体字段：

| 字段         | 类型      | 必填 | 说明                   |
|------------|---------|----|----------------------|
| deviceName | string  | 是  | 目标设备名称（路由到对应 Mapper） |
| namespace  | string  | 否  | 命名空间（缺省 default）     |
| property   | string  | 是  | 目标属性名（如 targetTemp）  |
| value      | float64 | 否  | 期望值（数值型）             |

响应示例（HTTP 200，边缘已确认且期望值已写入云端设备状态）：

```
{"status":"ok","acked":true}
```

下发成功后，通过 GET /api/v1/devices 可在 desired 字段中看到写入的期望值。

**命名空间路由（v0.2.0）：**路由键为 namespace/deviceName——同名设备可按命名空间隔离共存（边缘注册表按 ns 隔离）；指令按 ns 路由到对应 Mapper，ns 不匹配任何已注册 Mapper 时边缘拒绝 → 502。设备所属 ns 由 Mapper 侧三级解析决定（WithNamespace 选项 > EDGEFLOW\_MODBUS\_NAMESPACE 环境变量 > 默认 default，详见第 6 章）。

#### 4.7.4 八态响应语义

三个下发端点统一使用以下响应语义（v0.7.0 起扩展为八态，新增 202/409/422/429）：

| 状态码 | 响应体 (示例)                                                     | 含义                                                                       |
|-----|--------------------------------------------------------------|--------------------------------------------------------------------------|
| 200 | <code>{"status":"ok","acked":true}</code>                    | 边缘已确认 (Ack ok)                                                           |
| 202 | release 对象 (status=pending)                                  | 已受理 (异步执行, v0.7.0) : 发布创建/回滚置位, 结果以 release 对象回读为准                       |
| 400 | <code>{"error":"operation and pod.name are required"}</code> | 请求非法: JSON 解析失败 / 缺必填字段 / operation 或 kind 不在白名单 / 资源格式非法或 request>limit |
| 404 | <code>{"error":"node offline or not registered"}</code>      | 节点未注册或离线 (消息未送达, 无需重试)                                                   |
| 409 | <code>{"error":"EDGEFLOW_RESOURCE_EXHAUSTED: ..."}</code>    | 冲突: podsync 资源超卖被拒 (不落盘不建容器); 模型 API 冲突族 (详见 §4.11.3)                    |
| 422 | <code>{"error":"...", "unknownNodes":[...]}</code>           | 语义不可执行 (v0.7.0): 发布目标版本非 active / 无 Ready 节点 / 白名单含未知节点 / 无 Pre-vActive  |
| 429 | <code>{"error":"ocsp rate limit exceeded"}</code>            | 限流: /ocsp per-IP 请求超限 (默认 10 req/s, burst 20)                            |
| 500 | <code>{"error":"send failed"}</code>                         | 其他内部错误                                                                   |
| 502 | <code>{"error":"edge rejected ack"}</code>                   | 边缘明确拒绝: 消息已送达但处理失败 (重试无意义)                                               |
| 504 | <code>{"error":"ack timeout after retries"}</code>           | 确认超时且重试耗尽: 可能已送达但未确认 (可重试, 边缘侧有幂等去重)                                     |

语义区分 (记忆要点): **202** = 已受理异步执行, 以 release 回读为准; **404** = 没送达 (无需重试); **409** = 送达但超卖/冲突拒绝 (重试无意义); **422** = 可执行性不满足 (业务前置); **429** = 限流 (稍后重试); **502** = 送达但被拒绝 (重试无意义); **504** = 可能送达但未确认 (可重试)。

## 4.8 监控指标

### 4.8.1 GET /metrics

用途：输出 Prometheus 文本格式指标，供 Prometheus 等监控系统抓取，观测云端运行状态。

```
curl http://127.0.0.1:8080/metrics
```

指标说明：

| 指标名                                             | 类型      | 含义                                                                          |
|-------------------------------------------------|---------|-----------------------------------------------------------------------------|
| edgeflow_cloudcore_nodes_total                  | gauge   | 已注册边缘节点总数<br>(含离线)                                                          |
| edgeflow_cloudcore_pods_total                   | gauge   | 云端 Pod 状态记录总数                                                               |
| edgeflow_cloudcore_devices_total                | gauge   | 云端设备状态记录总数                                                                  |
| edgeflow_cloudcore_http_requests_total          | counter | 云端 HTTP 请求累计数<br>(按路由模式与状态码分桶)                                              |
| edgeflow_cloudcore_active_connections           | gauge   | CloudHub 活跃连接数                                                              |
| edgeflow_cloudcore_lease_renewal_failures_total | counter | 外部 etcd 心跳租约续约失败累计数 (v0.8.0; 持续增长 = etcd 侧异常/网络分区, 告警阈值按判活 TTL 折算; 仅外部模式输出) |

注：v0.7.0 起模型 API 调用同样计入 edgeflow\_cloudcore\_http\_requests\_total 分桶 (按路由模式与状态码)；v0.8.0 起指标 5→7 项 (新增续约失败计数, 0 值也输出便于面板基线)。

## 4.9 OCSP 在线吊销查询 (RFC 6960)

### 4.9.1 POST /ocsp

用途：标准 OCSP (Online Certificate Status Protocol, RFC 6960) 应答端点，供外部工具在线查询边缘节点证书的吊销状态。证书吊销后，除 mTLS 握手按 CRL (证书吊销列表) 拒收外 (见第 8 章证书管理)，外部查询也可通过本端点进行。

```
curl -X POST http://127.0.0.1:8080/ocsp \
-H 'Content-Type: application/ocsp-request' \
--data-binary @request.der
```

协议要点:

- 请求体为 DER 编码的 OCSPRequest (Content-Type: application/ocsp-request), 响应为 DER 编码的 OCSPResponse (application/ocsp-response), 由云端 CA 私钥签名, 响应自带可信性;
- 请求体上限 16 KiB (标准 CertID 请求远小于该值), 超限返回 400;
- 免认证端点, 以 per-IP 令牌桶限流防滥用: 默认 10 req/s、burst 20, 超限返回 429; 速率可经环境变量 EDGEFLOW\_CLOUDCORE\_OCSP\_RATE\_LIMIT 调整;
- 成功响应带 Cache-Control: max-age=3600 (响应 nextUpdate  $\approx$  7 天);
- 吊销状态与 CRL 同源 (均读 crl.json), 证书被吊销后 CRL 与 OCSP 双通道同时生效;
- 状态码: 200 返回签名后的吊销状态响应 (good/revoked/unknown); 400 请求超限、为空或 DER 损坏; 429 触发 per-IP 限流; 500 CA 不可用、吊销记录损坏或响应构建失败 (fail-closed, 不静默放行);
- 客户端查询库新增新鲜度校验入口 OCSPStatusAtWithPolicy/ParseOCSPResponseWithFreshness (fail-closed: 过期或未来时间均拒绝, 默认 5 分钟时钟 skew 容忍); 旧入口 (OCSPStatus/OCSPStatusAt) 行为不变; 生产路径接入 OCSP 客户端时须使用 WithPolicy 入口 (当前边缘 mTLS 握手仍按 CRL 校验)。

## 4.10 API 认证

云端 API 认证默认关闭; 生产环境建议开启。

### 4.10.1 启用认证

启动 cloudcore 前设置两个环境变量:

```
EDGEFLOW_CLOUDCORE_AUTH=on EDGEFLOW_CLOUDCORE_API_TOKEN=*** ./bin/cloudcore
```

注意: EDGEFLOW\_CLOUDCORE\_AUTH=on 时必须同时设置 EDGEFLOW\_CLOUDCORE\_API\_TOKEN, 否则 cloudcore 拒绝启动。

### 4.10.2 调用方式

认证开启后, 所有 /api/v1/\* 请求必须携带请求头 Authorization: Bearer <token>:

```
curl -H 'Authorization: Bearer <你的令牌>' http://127.0.0.1:8080/api/v1/nodes

# 未携带或携带错误 token 时返回 401
curl http://127.0.0.1:8080/api/v1/nodes
```

说明：`/healthz`、`/metrics` 与 `/ocsp` 不参与认证，始终可访问，以保证探活、监控采集与在线吊销查询通道可用（`/ocsp` 响应由 CA 签名，自带可信性，并以 per-IP 限流防滥用，见上文）。v0.7.0 新增的 17 个模型 API 注册于同一 `/api/v1/*` 路由树，认证开启时同样自动要求 Bearer Token（401 fail-fast），审计台账自动记录。

#### 4.11 模型 API (v0.7.0: 模型仓库 / 版本管理 / 灰度发布)

v0.7.0 新增 17 个端点（总端点 14→31，见 §4.2）：模型 5 + 版本 6（CRUD 4 + activate/archive）+ 发布 5（创建/列表/详情/取消/回滚）+ 部署影子 1。全部注册于既有 `apiMux`，自动挂 `auth.Middleware`（Bearer Token，默认 off 向后兼容）与 `ledger.Middleware`（审计台账）——鉴权/审计零新代码；既有 14 端点一行不改。

核心概念：“**镜像即模型、Tag 即版本**”——平台不托管镜像实体，只登记镜像引用（mirror）+ sha256 摘要防篡改，镜像实体仍在客户镜像仓库；发布器复用既有 `podsync`（镜像 Pod）+ `config-sync`（模型版本/参数 ConfigMap）经可靠投递下发。云边协议无新消息类型，边缘零代码改动，旧版 `edgecore` 直接可用。

数据模型与状态机设计见 `docs/ARCHITECTURE.md` 决策 R16；实现包为 `cloud/pkg/modelrepo`（台账/校验/存储）+ `cloud/pkg/modelrelease`（灰度控制器/算法/部署执行器）。请求体上限 1 MiB（超限 400 request body too large）。

##### 4.11.1 对象模型与键空间

五类对象：

| 对象                | 语义                     | 关键字段                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model             | 模型台账（一级对象，模型名唯一）       | name<br>( <code>^[A-Za-z0-9][A-Za-z0-9._-]{0,63}</code> 禁 /)、description ( $\leq 256$ )、type ( $\leq 64$ , 开放字符串)、metadata (键 $\leq 64$ / 值 $\leq 1024$ )、createdAt/updatedAt (Unix 毫秒)                                                                                                                                                                        |
| ModelVersion      | 版本台账（“Tag 即版本”）        | version (字符集同 name, 模型内唯一)、mirror (镜像 ref 必带 tag)、sha256 ( <code>^sha256:[0-9a-f]{64}\$</code> , 存储统一小写)、sizeBytes ( $\geq 0$ )、archs (白名单 amd64/arm64, 空 = 不限制)、status (draft/active/archived)、metadata (模型参数/阈值, 发布时平铺进 config-sync)                                                                                                                         |
| ModelRelease      | 灰度发布任务（异步执行）           | id (UUID)、model/version (目标版本, 创建时须 active)、target (nodeIDs 白名单   percentage 1..100)、targetNodes (创建时物化的有序节点快照, 运行期不重算)、batchSize ( $\geq 1$ , 默认 1)、pauseBetween ( $\geq 0$ ms, 默认 0)、failFast (默认 true)、prevActive (回滚目标; 无则 "")、status (pending/running/succeeded/failed/canceled/rolled_back)、nextBatchAt/createdAt/startedAt/finishedAt、rollbackRequested |
| NodeReleaseResult | 逐节点执行结果（release 键下独立键） | nodeID、status (pending/deployed/failed/skipped)、version (该节点被部署到的版本)、reason (failed 原因)、batch (批次数, 1 起)、startedAt/finishedAt                                                                                                                                                                                                                                  |
| DeploymentState   | 部署影子（跨发布全局视图）          | model、version、mirror、releaseID、updatedAt                                                                                                                                                                                                                                                                                                                       |

**键空间**(新增前缀 /edgeflow/models/, 与既有键完全隔离; /edgeflow/\_meta/schemaVersion 不 bump):

- meta/<model> —— 模型台账;
- versions/<model>/<version> —— 版本台账;
- guards/<model> —— 在途发布守卫 (值 = releaseID);
- releases/<releaseID> —— 发布 head (状态机 CAS 键);
- releases/<releaseID>/nodes/<nodeID> —— 逐节点结果 (perNode);
- releases/<releaseID>/lock —— 发布领跑锁租约键 (TTL 默认 60s);
- deployments/<model>/<nodeID> —— 部署影子。

**版本状态机** (Activate/Archive API 驱动):

```
draft --activate--> active --archive--> archived
|           ^           |
+---delete-->(删)      | (激活时自动降级旧 active) +---delete-->(删)
```

- **activate**: 仅 draft→active; **自动把当前 active 版本置为 archived** (两键 CAS 序列 + 失败补偿); archived 不可再激活;
- **archive**: 仅 active→archived; 存在指向该版本的 pending/running 发布 → 409;
- **delete 版本**: 仅 draft/archived (active → 409, 先归档或删除模型);
- **发布/回滚不改变版本状态**: 发布要求目标 active (只读校验); 回滚可部署 archived 的 prevActive (台账状态不变, 由调用方按需显式 activate)。

**发布任务状态机** (控制器 + API 协作, head 键 CAS):

```
pending -> running -> 全部 deployed -----> succeeded
      |- fail-fast 中止 / 存在失败(且跑完) -> failed
      |- cancel (批次边界生效) -----> canceled
      +- rollback 置位 -> 逆序执行 -----> rolled_back

(成功/失败/取消均可再 rollback; 终态后 guard 释放, 同模型可再次发布)
```

#### 4.11.2 端点明细与请求示例

**POST /api/v1/models —— 创建模型**



```
curl -X POST http://127.0.0.1:8080/api/v1/models \
-H 'Content-Type: application/json' \
-d '{"name":"defect-detector","description":"缺陷检测模型","type":"detection",
  "metadata":{"owner":"qa-team"}}'
```

请求体字段（其余写端点的可选字段语义一致，不重复列表）：

| 字段          | 类型                | 必填 | 说明                                                            |
|-------------|-------------------|----|---------------------------------------------------------------|
| name        | string            | 是  | <code>^[A-Za-z0-9][A-Za-z0-9._-]{0,63}\$</code> ；重复 → 409     |
| description | string            | 否  | ≤256 字符                                                       |
| type        | string            | 否  | 开放字符串 ≤64（推荐 classification/detection/segmentation/llm/other） |
| metadata    | map[string]string | 否  | 键 ≤64、值 ≤1024；键匹配 <code>^[A-Za-z0-9._-]+\$</code>             |

响应 200：完整 Model 对象（含 createdAt/updatedAt）。PUT 允许改 description/type-/metadata（metadata 整表替换）；name/createdAt 不可变。DELETE 前置：无 active 版本、无在途发布（否则 409），级联删除 draft/archived 版本 + 部署影子（非事务，见 KNOWN-ISSUES L26）。

POST /api/v1/models/{modelName}/versions —— 登记版本

```
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/versions \
-H 'Content-Type: application/json' \
-d '{"version":"v1.2.0","mirror":"registry.example.com/edgeflow/models/defect-
  detector:v1.2.0","sha256":"sha256:9
  f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08","sizeBytes
  ":482344960,"archs":["amd64","arm64"],"metadata":{"threshold":"0.8","
  batchSize":"32"}}'
```

请求体字段：

| 字段        | 类型                | 必填 | 说明                                                                 |
|-----------|-------------------|----|--------------------------------------------------------------------|
| version   | string            | 是  | 字符集同 name；同模型重复 → 409                                              |
| mirror    | string            | 是  | 镜像 ref， <b>必须带 tag</b> ；整体匹配镜像 ref 正则（禁 ../连续 //空白；tag 由最后一个 : 界定） |
| sha256    | string            | 是  | <code>^sha256:[0-9a-f]{64}\$</code> （大小写不敏感接受，存储统一小写）              |
| sizeBytes | int64             | 否  | ≥0；缺省 0                                                            |
| archs     | []string          | 否  | 元素 ∈ {amd64, arm64} 白名单，去重；空 = 不限制（多架构语义）                          |
| metadata  | map[string]string | 否  | 模型参数/阈值；发布时平铺进 config-sync（见 §4.11.4）                              |

响应 200：完整 ModelVersion（status=draft）。（POST 不激活——激活是显式动作。）

**POST /api/v1/models/{modelName}/versions/{version}/activate —— 激活版本**

```
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/versions/v1.2.0/activate
```

用途：draft→active，自动把当前 active 版本置为 archived；archived 不可再激活（409）。发布目标版本必须为 **active**。归档端点 (.../archive, active→archived) 同属此状态机。

**POST /api/v1/models/{modelName}/releases —— 创建灰度发布（202 异步受理）**

目标二选一：按节点白名单（nodeIDs）或按比例（percentage）：

```
# 形态一：按比例 25%，batchSize=2、批间暂停 30s、fail-fast 开
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/releases \
-H 'Content-Type: application/json' \
-d '{"version":"v1.2.0","target":{"type":"percentage","percentage":25},"batchSize":2,"pauseBetween":30000,"failFast":true}'

# 形态二：白名单 2 台
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/releases \
-H 'Content-Type: application/json' \
-d '{"version":"v1.2.0","target":{"type":"nodeIDs","nodeIDs":["edge-node-1","edge-node-2"]},"batchSize":1,"pauseBetween":0,"failFast":true}'
```

请求体字段：

| 字段                | 类型       | 必填   | 说明                                                                                                                |
|-------------------|----------|------|-------------------------------------------------------------------------------------------------------------------|
| version           | string   | 是    | 目标版本；须为 <b>active</b> （否则 422）                                                                                    |
| target.type       | string   | 是    | nodeIDs / percentage 二选一                                                                                          |
| target.nodeIDs    | []string | 条件必填 | 白名单；元素须匹配 <code>^[A-Za-z0-9._-]+\$</code> 且全部已注册（否则 422，响应 <code>"unknownNodes":[...]</code> ）；允许离线节点入列（运行时按离线处理） |
| target.percentage | int      | 条件必填 | 1..100（越界 400）；分母 = 创建时刻 Ready 节点（0 台 Ready → 422）                                                                |
| batchSize         | int      | 否    | ≥1，默认 1                                                                                                           |
| pauseBetween      | int64    | 否    | 批间暂停毫秒，≥0，默认 0                                                                                                    |
| failFast          | bool     | 否    | 默认 true                                                                                                           |

创建前置校验（依序）：模型/版本存在（404）→ 目标 active（422）→ target 格式（400）→ 白名单节点已注册 / percentage 合法（422/400）→ 物化 TargetNodes → 确定 prevActive → guard CAS + release 键 CAS（同模型在途 → 409 含在途 releaseID；孤儿 guard 自愈）→ perNode 全部 pending 预写。

响应 **202**：完整 ModelRelease 对象（status=pending，targetNodes 已物化，perNode 汇总 pending=N）。

按比例分母口径与舍入（创建时刻快照）：

| readyCount | 规则                                                                                                | 例                                                |
|------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------|
| 0          | <b>422 拒绝创建</b> （no ready nodes）                                                                  | —                                                |
| 1          | $n = 1$ （任何 pct 均落该节点）                                                                            | 1 台 → 1                                          |
| $\geq 2$   | $n = \text{ceil}(\text{readyCount} \times \text{pct} / 100)$ ，且 $1 \leq n \leq \text{readyCount}$ | 23×10%→3；<br>3×50%→2；<br>100×1%→1；<br>10×100%→10 |

- 节点选择确定性：Ready 名单按 NodeID 字典序取前 n（跨副本可复现）；archs 非空时先按 `node.arch ∈ version.archs` 过滤再取前 n（过滤后 0 台 → 422，文案含 `no ready nodes for archs [...]`）；白名单模式不做 arch 过滤；
- 目标集合以创建时快照为准：运行期节点掉线/新节点加入不重算；不同模式/迁移后的百分比目标集合不跨模式可比（§4.11.6）。

GET .../releases ——发布列表与详情

```
# 发布列表（按 createdAt 升序）
curl http://127.0.0.1:8080/api/v1/models/defect-detector/releases

# 发布详情（含 perNode 汇总）
curl http://127.0.0.1:8080/api/v1/models/defect-detector/releases/<releaseID>
```

列表响应为 K8s List 风格(`{"kind": "ModelReleaseList", "apiVersion": "v1", "items": [...]}`，空为 `[]`)，对齐 PodStatusList/DeviceStatusList。发布详情额外返回派生汇总：

```
{"summary": {"total": 10, "deployed": 7, "failed": 0, "pending": 3, "skipped": 0}}
```

（summary 现算，非冗余存储。）

POST .../releases/{releaseID}/cancel ——取消发布

```
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/releases/<releaseID>/cancel
```

响应 200 + release (status=canceled)；取消在**批次边界**生效，未执行节点 ≤1 扫描周期补齐 skipped (KNOWN-ISSUES L27)。终态 release cancel → 409 (`{"error": "release already <status>"}`)。

POST .../releases/{releaseID}/rollback ——回滚发布（202 异步）

```
curl -X POST http://127.0.0.1:8080/api/v1/models/defect-detector/releases/<
  releaseID>/rollback
```

- **前置校验**: `status`  $\in$  {running, succeeded, failed, canceled} (pending/rolled\_back  $\rightarrow$  409); `prevActive`  $\neq$  "" 且版本存在(否则 422 no previous active version); `release.version` == 模型当前 active 版本 (已被更新版本接管  $\rightarrow$  409, 文案引导显式 activate 目标旧版本或发起新发布, KNOWN-ISSUES L27);
- 通过  $\rightarrow$  202 + release (rollbackRequested=true); 控制器**逆序**逐批执行(批间 pause=0), 失败不回滚中止(能回多少回多少, perNode 明细可查, KNOWN-ISSUES L24); 完成  $\rightarrow$  rolled\_back;
- **执行期复查**: runRollback 起始重读版本表——若执行前已被新版本接管或 prevActive 被删  $\rightarrow$  中止: head=failed (reason 明确) + 清除 rollbackRequested (防活锁) + 未执行节点标 skipped; API 端 202 照旧, 结果以 head 终态回读为准。

## GET /api/v1/models/{modelName}/deployments ——部署影子查询

```
curl http://127.0.0.1:8080/api/v1/models/defect-detector/deployments
```

部署影子 = 云端期望态(对标设备 Desired), 提供“版本—节点—时间”追踪(F41 台账): 每项 {"model":..., "version":..., "mirror":..., "releaseID":..., "updatedAt":...}, 键为 /edgeflow/models/deployments/<model>/<nodeID>。写入时机: podsync + config-sync 均 acked 后; 写穿失败  $\rightarrow$  日志 Warn (下发已生效, 仅影子视图缺该记录, release/perNode 已持久化不受影响)。与边缘实际运行版本(PodStatus 上报)分离; 重启后从 etcd 恢复(embed/外部), 纯内存模式重启丢失(KNOWN-ISSUES L22)。影子为派生台账整值覆盖、无 CAS 需求——与 Desired (权威期望态, modRevision CAS) 的差异为有意设计。模型删除  $\rightarrow$  级联删除该模型部署影子。



4.11.3 错误语义

| 状态码 | 语义                                                                   |
|-----|----------------------------------------------------------------------|
| 200 | 成功（写类端点返回完整对象或 {"status":"ok"} 形态）                                   |
| 202 | 已受理（异步执行）：发布创建 / 回滚置位                                                |
| 400 | 请求非法：JSON 解析失败 / 缺必填 / 字符集或枚举越界 / percentage 越界 / batchSize<1 / 请    |
| 404 | 模型/版本/发布/部署影子不存在； <b>模型不存在时其子资源一律 404</b>                            |
| 409 | 冲突：模型/版本已存在；删除 active 版本；归档/激活状态机非法；同模型在途发布（含在途 r                    |
| 422 | <b>语义不可执行</b> ：发布目标版本非 active / 无 Ready 节点 / 白名单含未知节点 / 无 PrevActive |
| 500 | 内部错误兜底（存储/序列化异常）                                                     |

响应体统一 `{"error": "< 机器可读原因 >", ... 上下文字段}` (409 发布冲突带 `"releaseID"`, 422 白名单带 `"unknownNodes": [...]`)。鉴权/审计链与既有端点完全一致 (401/审计自动覆盖)。

#### 4.11.4 config-sync 载荷约定 (边缘模型版本感知, 零边缘代码)

发布者对每目标节点自动执行两步: `podsync add`——Pod 名 `edgeflow-model-<sanitized>` (`sanitize(name) = 小写 + .→-`); namespace 固定 `edgeflow`; `image = 版本镜像`; `replicas=1`——模型实例多副本由用户后续自行 `podsync` 编排, 发布语义 = “该版本上机”; `config-sync add`——ConfigMap, 同命名约定。两步均 `acked` → 部署影子写穿 (§4.11.2)。

**ConfigMap 载荷约定** (`configs/edgeflow/edgeflow-model-<sanitized>`, 保留键由发布者保证):

```
{
  "model":      "defect-detector",
  "version":    "v1.2.0",
  "mirror":     "registry.example.com/edgeflow/models/defect-detector:v1.2.0",
  "sha256":     "sha256:9f86...",
  "type":       "detection",
  "releasedAt": "1787000000000"
}
```

- 保留键 6 个: `model / version / mirror / sha256 / type / releasedAt`;
- `version.metadata` 全部键值平铺追加进 `data` (模型参数随版本走); 与保留键冲突 → 保留键优先 + 控制器日志 Warn;
- 推理容器挂载/读取该 ConfigMap 即得“当前模型版本与参数”; 版本切换 = 下一次 `config-sync` 覆盖 (EdgeHub 幂等去重 + MetaManager SQLite 落盘保证重启后仍是新版本元数据);
- 发布回滚同样经此通道把 `version` 字段改回 `prevActive`——边缘无状态机依赖, 纯声明式收敛;
- 错误映射 (`perNode.Reason` 文案): `node offline or not registered / ack timeout after retries / edge rejected ack / send failed: <err>`。

#### 4.11.5 灰度运营建议

1. 节奏: 先 1 台试点 → 小批 → 全量。试点用白名单 (`target.type=nodeIDs`, 1 台) 验证镜像可用与推理结果; 小批用 `batchSize=1~2 + pauseBetween ≥ 30s` 留观察窗口; 验证通过后放大比例或白名单; 确认无误后 `percentage=100`;
2. fail-fast 保持默认开 (`true`): 单节点失败立即中止并标记剩余节点 `skipped`, 避免坏版本扩散; 需要“看完全部节点失败情况”时再显式关;

3. 回滚前置条件 = “该发布版本仍是模型当前 active 版本”（未被新版本接管）；已被接管 → 409，先显式 activate 目标旧版本（或发起新发布）再回滚。回滚为逆序批量执行、失败不回滚中止（能回多少回多少，perNode 明细可查）；
4. 发布成功 ≠ 镜像可用：平台下发的是声明（mirror ref），拉取发生在边缘；镜像仓库不可达/镜像损坏会在边缘 PodStatus 暴露（CrashLoop 等），发布任务本身仍可能 succeeded——发布前建议在试点节点确认 Pod Running；
5. 耗时口径：批内逐节点串行（每节点 2 次可靠下发，最坏 ~10s+/节点超时）；batchSize 控制批粒度/暂停节奏，不是并发度；大 fleet 全量耗时 =  $\Sigma(\text{单节点部署耗时}) + \Sigma(\text{pause})$ ；
6. 模型实例多副本：发布语义 = “该版本上机”(replicas=1)；多副本由用户后续自行 podsync 编排。

4.11.6 三模式行为矩阵

发布控制器在三类存储模式下同一实现、同一行为口径：

| 能力               | 纯内存<br>(ETCD_ENABLED=false)<br>且无 END-POINTS) | embed etcd (默认) | 外部 etcd (多副本)          |
|------------------|-----------------------------------------------|-----------------|------------------------|
| 模型/版本/发布 CRUD    | 内存（mutex 串行）                                  | 写穿持久化           | 写穿 + CAS + watch       |
| release 任务执行     | 正常工作                                          | 正常              | 领跑锁 + 接管               |
| release/部署影子重启恢复 | 重启丢失<br>(KNOWN-ISSUES<br>L22 登记，明示)           | etcd 恢复         | etcd 恢复                |
| 发布领跑锁            | 单实例恒成功（逻辑空转）                                  | 同左              | 租约锁 + 接管<br>(≤TTL 60s) |
| 并发安全             | 单进程                                           | 单副本 CAS 恒成功     | CAS + guard + 锁功       |
| 部署影子恢复           | 重启清空                                          | 恢复              | 恢复                     |

**模式差异补注：**不同模式/迁移后百分比发布的目标集合**不跨模式可比**——百分比分母 = 创建时刻 Ready 节点（embed/外部模式的 Ready 判定口径不同），且目标集合在创建时物化为快照；以创建时快照为准，跨模式迁移后重新发起发布。

发布相关环境变量（v0.7.0，全部可选）：



| 环境变量                                     | 默认  | 说明                                                                                                                 |
|------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------|
| EDGEFLOW_CLOUDCORE_RELEASE_SCAN_INTERVAL | 5s  | 发布控制器扫描周期 (>0, 非法 fail-fast)                                                                                       |
| EDGEFLOW_CLOUDCORE_RELEASE_LOCK_TTL      | 60s | 发布领跑锁租约 TTL ( $\geq 15s$ , 非法 fail-fast) ; 刷新周期 = $\max(5s, TTL/3)$ ; 仅外部模式消费, embed/纯内存显式设置 $\rightarrow$ Warn 忽略 |

## 4.12 排障指南

| 现象                           | 处理                                                                                                                                                                                                                   |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 返回 401                       | API 认证已开启但未携带或携带错误的 token；确认请求头 <b>Authorization: Bearer &lt;token&gt;</b> 正确                                                                                                                                        |
| 返回 400                       | 请求体 JSON 非法或缺少必填字段、operation/kind 不在白名单；按响应中 <b>error</b> 提示修正                                                                                                                                                       |
| 返回 404（下发类）                  | 节点未注册或离线，消息未送达；先确认 edgecore 运行状态与节点 ID 拼写，无需重复重试                                                                                                                                                                     |
| 返回 502                       | 消息已送达但被边缘拒绝；检查请求体是否符合边缘校验规则（如设备名、属性名是否正确、namespace 是否匹配已注册 Mapper）                                                                                                                                                   |
| 返回 504                       | 可能已送达但未确认（超时重试耗尽）；可稍后重试，边缘侧有幂等去重，不会重复执行                                                                                                                                                                              |
| 返回 409                       | 两类场景：podsync 超卖拒绝（响应 <b>error</b> 为 <b>EDGEFLOW_RESOURCE_EXHAUSTED</b> ，不落盘不建容器，重试无意义）；模型 API 冲突族（模型/版本已存在、删除 active 版本、状态机非法、同模型在途发布、CAS 冲突耗尽、回滚被新版本接管）——按响应 <b>error</b> 文案与上下文文字段定位（发布冲突 409 带 <b>releaseID</b> ） |
| 返回 422                       | 模型发布业务前置不满足：目标版本非 active、无 Ready 节点、白名单含未知节点（响应带 <b>unknownNodes</b> ）、无 PrevActive 可回滚——按提示修正后重新创建发布                                                                                                                |
| 发布创建/回滚返回 202                | 异步任务：用 <b>GET /api/v1/models/{modelName}/releases/{releaseID}</b> 回读状态与 perNode 汇总（pending→running→终态），不要以 202 视为执行完成                                                                                                |
| 日志出现 <b>concurrent-write</b> | 外部多副本形态下多副本并发写同一设备 Desired，CAS 冲突重试耗尽属预期语义（HTTP 仍 200）；重发指令即可收敛                                                                                                                                                      |
| 云端重启后查询为空                    | v0.4.0 起分级持久化：注册台账与设备 Desired 跨重启保留（节点重启后短暂 Unknown，边缘 ≤1 心跳周期翻新 Ready，无需重新注册）；Pod 状态与上报属性短暂清空（≤1 上报周期自愈），非永久丢失                                                                                                      |

## 第5章 边缘节点与自治运行

EdgeFlow 的边缘侧由一个常驻进程 **edgecore**（边缘核心）承载。它运行在边缘节点上，负责三件事：与云端建立并维持管理通道、按云端期望状态调谐本机容器、以及在本机维护设备影子与元数据。本章说明 **edgecore** 的接入、通信、自治行为，以及如何配置它。

### 5.1 接入与连接管理

#### 5.1.1 注册

**edgecore** 启动后第一条消息是 **Register**（注册），云端 CloudHub 校验后回 **RegisterAck**（注册确认）。注册通过后，节点在云端注册表中可见，状态为 **Ready**。节点 ID 由环境变量 **EDGEFLOW\_EDGECORE\_NODE\_ID** 或配置文件（**config/edgecore.json** 的 **nodeID** 字段）指定，环境变量优先；均未设置时使用默认约定 **edge-<主机名>**。

```
# 指定节点 ID 与云端地址启动 edgecore
export EDGEFLOW_EDGECORE_NODE_ID=edge-worker-01
export EDGEFLOW_EDGECORE_CLOUD_ADDR=ws://192.168.1.10:10000/v1/edge
bin/edgecore
```

#### 5.1.2 心跳保活

注册完成后，**edgecore** 以 **30 秒** 为周期向云端发送心跳（Heartbeat）。云端 **NodeController** 每 **30 秒** 扫描一次注册表，若某节点心跳停滞超过 **180 秒**（约 6 个心跳周期），将该节点标记为 **Offline**。因此：

- 心跳周期：固定 **30s**（云边契约常量，不可配置）；**EDGEFLOW\_EDGECORE\_REPORT\_INTERVAL** 仅控制 Pod 状态上报周期；
- 云端超时阈值：**EDGEFLOW\_CLOUDCORE\_NODE\_TIMEOUT**，默认 **180s**；
- 云端扫描周期：**EDGEFLOW\_CLOUDCORE\_NODE\_SCAN\_INTERVAL**，默认 **30s**。

**判活机制差异(v0.6.0)**: 外部 **etcd** 多副本模式下，云端 **NodeController** 停用，不再参与判活；判活的唯一事实源改为 **etcd** 租约视角的心跳键（**/edgeflow/registry/heartbeats/<nodeID>**）：每次心跳 **Grant+Put** 一个新租约，租约到期自动删除心跳键即判 **Offline**（软离线）。该模式下：

- 判活阈值：**EDGEFLOW\_CLOUDCORE\_NODE\_LEASE\_TTL**，默认 **300s**（低于 **90s** 打印告警，非法值启动即失败）；
- **EDGEFLOW\_CLOUDCORE\_NODE\_TIMEOUT** 不再参与判活（显式设置仅告警忽略）；
- **EDGEFLOW\_CLOUDCORE\_NODE\_SCAN\_INTERVAL** 迁用为心跳键重扫/GC 周期（默认 **30s**）。

嵌入式 etcd 与纯内存模式维持上文表述 (NodeController 扫描 30s、超时 180s 判活)。

### 5.1.3 通道压缩

云边通道的 gzip 压缩**默认开启**: edgecore 在 Register 消息中声明压缩能力 (compression: "gzip"), 云端在 RegisterAck 中回带确认, 协商成功后双向启用。config/cloudcore.json 中设置 compress:false 可关闭: 云端不再回带压缩能力, 边缘自动回落明文传输, 与旧版本互操作兼容。

### 5.1.4 断线重连

网络抖动或云端重启导致连接断开时, edgecore **自动重连**, 无需人工干预:

- 指数退避: 1s/2s/4s/..., 上限 60s;
- 重连成功后重新执行注册流程, 退避计时重置;
- 重连期间本地功能不受影响 (见第 五 章第 5.4 节)。

## 5.2 可靠投递

云边之间的下行消息 (Pod 配置、设备指令等) 使用**可靠投递**语义, 保证云端能确认消息确实被边缘处理:

1. 云端发送消息并登记到 **pending** (待确认) 队列;
2. 边缘收到并处理成功后回 **Ack**;
3. 云端收到 Ack 后移出 pending 队列;
4. 超时 (默认单次 5 秒) 未确认, 使用**同一消息 ID** 重试, 最多 3 次; 仍失败则向 API 调用方返回 504。

边缘侧对重复消息做**幂等去重**: 同一消息 ID 处理过一次后不再重复执行, 因此重试不会造成重复下发。API 层表现见第 九 章与附录 B。

## 5.3 Edged 应用调谐

**Edged** 是 edgecore 内置的容器运行时管理模块, 对标 KubeEdge 的 Edged。它采用**声明式调谐**模型: 云端下发期望状态 (Pod 定义) 后, Edged 周期性比对期望与实际情况, 差异即收敛。

- 调谐周期: EDGEFLOW\_EDGECORE\_RECONCILE\_INTERVAL, 默认 5s;
- 启动时立即调谐一次, 之后按周期循环;
- 调谐数据来自 MetaManager 落盘数据 (SQLite), 因此断网时调谐照常进行。

### 5.3.1 多副本 Pod

云端下发的 Pod 支持 **replicas** 多副本。Edged 每轮调谐将本机实际运行的副本数收敛到期望值：

- 实际副本 **少于**期望 → 逐个补齐拉起；
- 实际副本 **多于**期望 → 逐个清理收缩；
- 每轮调谐只处理一个副本的差异（批大小 1），逐轮滚动，避免瞬间抖动。

### 5.3.2 健康自愈与 CrashLoopBackOff

Edged 每轮调谐都会检查容器运行状态（**Inspect**）。容器非 **Running**（异常退出、被杀死等）时自动重启，无需人工介入：

- 每次重启累加 **RestartCount** 并上报云端；
- 连续异常重启达到 **3 次** 阈值后进入 **CrashLoopBackOff** 退避：退避时长 **30 秒**，退避期间不再反复拉起；
- 容器稳定运行 **60 秒**后重置计数，恢复正常自愈节奏。

典型表现：退出型镜像（如 **busybox** 执行完即退出）会被反复拉起并进入退避，属预期行为；业务容器持续崩溃时请查看容器日志定位根因（见第 [九](#) 章）。

### 5.3.3 镜像漂移检测

Edged 在调谐时会比**对期望镜像与实际运行镜像**：

- 已运行的容器镜像与期望不一致 → 判定为**镜像漂移**，自动重建：停止旧容器 → 用期望镜像重新拉起；
- 同一轮只重建 1 个漂移容器，逐轮滚动完成全部收敛；
- 该机制同时是滚动更新的基础：更新 Pod 定义中的镜像后，Edged 自动逐副本完成替换。

### 5.3.4 资源限制与漂移检测（v0.2.0）

云端下发 Pod 时可通过 **resources** 字段声明资源诉求（v0.2.0 起，K8s 风格，零值表示不限制）：

```
"resources": {
  "cpuRequest": "100m",
  "cpuLimit": "250m",
  "memoryRequest": "64Mi",
  "memoryLimit": "128Mi"
}
```

- **落地**：容器创建时映射为 `docker --cpus / --memory` 参数；`--memory-swap` 被禁用，容器无法用 `swap` 规避内存限制；
- **超卖准入** (`admitPodResources`)：边缘按节点容量（自动探测，可用 `NODE_CPU_MILLI / NODE_MEMORY_BYTES` 覆盖）与超卖率（默认 1.5）准入；超卖拒绝时容器**不落盘、不创建**，云端侧对应返回 `409 EDGEFLOW_RESOURCE_EXHAUSTED`；
- **资源漂移检测**：与镜像漂移并列——若外部（如 `docker CLI`）修改了运行中容器的资源限制，`Edged` 判定为**资源漂移**，自动 `stop+ 重建`；每轮最多重建 **1 个** 漂移容器，逐轮滚动收敛，重建不计入 `RestartCount`。

## 5.4 断网自治

**自治**是边缘计算的核心价值：云端不可达时，边缘业务不能停。

1. 云端断开后，已下发的容器**持续运行**，调谐、设备采集、本地指令照常工作；
2. 断网语义验证：自动化用例以 **60 秒短时断网**模拟验证；**30 分钟级**长时运行语义（M2 验收口径）需在真实环境长跑确认；
3. 重连成功后，`edgecore` 重新注册，并将离线期间的 Pod 状态、设备上报**自动同步**到云端，云端视图恢复一致。

注意：云端状态为**分级持久化**（v0.4.0 起）：节点注册台账与设备 `Desired` 跨重启保留，云端进程重启后节点**无需重新注册**——重启后节点短暂显示为 `Unknown`，边缘下一心跳周期内翻新为 `Ready`；Pod 状态与设备上报属性重启后短暂清空，**不超过 1 个上报周期**内由边缘周期上报自动补全自愈（非永久丢失）。边缘侧元数据由 `MetaManager` 持久化到本机 `SQLite`，断网期间不丢失。

## 5.5 edgecore 配置（环境变量 + 配置文件）

`edgecore` 采用**环境变量 + 配置文件**双轨配置，加载优先级从高到低为：**环境变量 > 配置文件 > 默认值**。配置文件为 JSON 格式，默认路径 `config/edgecore.json`（可用 `--config` 标志覆盖），所有字段可选（缺省回落默认值），支持 `cloudAddr`、`nodeID`、`podReportInterval`、`deviceReportInterval` 与 `reconcileInterval` 五个字段，例如：

```
{
  "cloudAddr": "ws://192.168.1.10:10000/v1/edge",
  "nodeID": "edge-worker-01",
  "podReportInterval": "30s",
  "deviceReportInterval": "30s",
  "reconcileInterval": "5s"
}
```

环境变量全集见表 5.1；时长类变量（如调谐/上报周期）合法范围为 **1s 至 10m**，超出范围或非法值回退默认值并打印告警。**敏感配置（接入令牌 EDGEFLOW\_EDGECORE\_TOKEN 等）** 仍**仅支持环境变量，不入配置文件**，防止令牌随文件分发泄露。

cloudcore 采用同构机制：配置文件 config/cloudcore.json（字段 port、hubPort、compress，全部可选）+ --config 覆盖；优先级为命令行 > 环境变量 > 配置文件 > 默认值。

表 5.1: edgecore 环境变量

| 变量                                       | 默认值                            | 说明                                       |
|------------------------------------------|--------------------------------|------------------------------------------|
| EDGEFLOW_EDGECORE_NODE_ID                | edge- < 主机名 >                  | 节点 ID，注册用                                |
| EDGEFLOW_EDGECORE_CLOUD_ADDR             | http://127.0.0.1:10000/v1/edge | 云端地址                                     |
| EDGEFLOW_EDGECORE_TLS                    | 关                              | on 时启用 mTLS（地址自动 wss://）                 |
| EDGEFLOW_EDGECORE_CERT_DIR               | Data/certs/                    | 证书目录                                     |
| EDGEFLOW_EDGECORE_DB_PATH                | 内置默认                           | MetaManager SQLite 路径                    |
| EDGEFLOW_EDGECORE_MQTT_ADDR              | tcp://127.0.0.1:1883           | MQTT broker 地址                           |
| EDGEFLOW_EDGECORE_RECONCILE_INTERVAL     | 5s                             | Edged 调谐周期                               |
| EDGEFLOW_EDGECORE_REPORT_INTERVAL        | 30s                            | Pod 状态上报周期                               |
| EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL | 30s                            | 设备状态上报周期                                 |
| EDGEFLOW_EDGECORE_MQTT_CONNECT_TIMEOUT   | 5s                             | 单次 MQTT 建连超时                             |
| EDGEFLOW_EDGECORE_TOKEN                  | 空                              | 接入令牌（keadm join 写入；仅环境变量，不入配置文件）         |
| EDGEFLOW_EDGECORE_NODE_CPU_CAPACITY      | 自动探测                           | 节点 CPU 容量（毫核），资源准入用（v0.2.0）              |
| EDGEFLOW_EDGECORE_NODE_MEMORY_CAPACITY   | 自动探测                           | 节点内存容量（字节，仅 Linux），资源准入用（v0.2.0）         |
| EDGEFLOW_EDGECORE_OVERCOMMIT_CPU_RATE    | 1.0                            | CPU 超卖率（v0.2.0）                          |
| EDGEFLOW_EDGECORE_OVERCOMMIT_MEMORY_RATE | 1.0                            | 内存超卖率（v0.2.0）                            |
| EDGEFLOW_EDGECORE_ENABLE_MAPPER          | true                           | Mapper 装配开关；false 时为纯影子模式（v0.2.0，见第 六 章） |

**周期类变量三态明示（v0.3.0）**：周期类环境变量（调谐/上报周期等）启动时打印状态日志，三态明示——取值合法（按值生效）、越界或非法（回落默认值并打印告警）、未设置（使用默认值）。

生产环境建议使用 keadm join 生成 edgecore.env 与 edgecore.service（systemd 单元），由安装脚本统一部署，避免手工拼写错误（见第 七 章升级回滚时的产物管理）。keadm join 产物与配置文件**双轨并存**：两者同时设置同一项时，以环境变量为准。

5.6 配置热重载

cloudcore 与 edgecore 均支持**配置热重载**，无需重启进程即可让部分配置生效：

- **触发方式**：向进程发送 **SIGHUP** 信号立即强制重载一次（不检查变更）；此外进程每 **60** 秒检查一次配置文件，mtime/size 变化即触发重载；多触发源串行化，并发安全；
- **重载失败保持旧配置** (fail-safe)：配置文件解析失败、文件缺失或端口绑定失败时，本次重载被整体拒绝，进程按旧配置继续运行。

不同配置项的热生效边界见表 5.2：

表 5.2: 配置热重载生效边界

| 组件        | 配置项                                      | 热重载行为                                                     |
|-----------|------------------------------------------|-----------------------------------------------------------|
| cloudcore | port (HTTP)                              | <b>热切换</b> ：先绑定新端口，成功后关闭旧监听；新端口绑定失败则拒绝本次重载，保持旧配置          |
| cloudcore | hubPort / compress                       | <b>需重启</b> ：热重载时打印告警并保持旧值（CloudHub 不支持运行期重建监听；压缩在连接注册时协商） |
| edgecore  | podReportInterval / deviceReportInterval | <b>热生效</b> ：重载后按新周期上报，无需额外动作                              |
| edgecore  | cloudAddr / nodeID / reconcileInterval   | <b>需重启</b> ：热重载时打印告警并保持旧值                                 |

生产建议：修改**需重启**类配置项后，在维护窗口重启对应进程使其生效；热生效类配置可即时下发，适合运行期调整上报频率。



## 第6章 设备接入

EdgeFlow 通过**设备影子** (DeviceTwin) 统一管理边缘设备：设备数据在边缘汇聚、周期上报云端；云端指令经边缘下发到设备。本章说明设备模型、指令链路、Mapper 框架（设备映射器）与数据面。

### 6.1 设备模型：影子

每台设备在边缘维护一份**影子** (Shadow / DeviceTwin)，包含两类属性：

**properties (实际值)** 设备实际上报的属性值（如温度、湿度），由 Mapper 采样写入，周期上报云端；

**desired (期望值)** 云端期望设备达到的属性值，通过指令下发写入，云端与边缘各存一份。

影子是边缘侧设备数据的**汇聚点**：无论设备通过何种协议接入 (MQTT、Modbus...)，云端看到的都是统一的影子视图。边缘默认每 `EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL` (30 秒) 把影子快照上报云端。

### 6.2 设备指令下发链路

云端下发指令走**可靠投递**通道（见第 五 章），完整链路为：

1. 调用方 POST `/api/v1/nodes/{nodeID}/device-command`, 携带 `deviceName` 与 `property` (期望值)；
2. 云端经 CloudHub 将 DeviceCommand 消息可靠送达边缘 (Ack 确认后 API 返回 200)；
3. 边缘按 `deviceName` 路由到对应 Mapper 直接执行指令 (MQTT 模式下另订阅本地指令主题 `devices/<ns>/<device>/command`，支持数据面直接下发)，设备属性发生变化；
4. 边缘影子 `properties` 更新，下一上报周期同步到云端。

```
# 下发设备指令：把 sensor-01 的目标温度设为 25
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/device-command \
-H 'Content-Type: application/json' \
-d '{"deviceName":"sensor-01","property":"targetTemp","value":25}'
# 期望响应: {"status":"ok","acked":true}
```

常见失败语义：节点离线 → 404（未送达，无需重试）；边缘拒绝 → 502（重试无意义）；确认超时 → 504（可重试，边缘幂等去重）。详见附录 B。

## 6.3 Mapper 框架

**Mapper**（设备映射器）负责把具体设备的协议翻译成 EdgeFlow 的统一设备模型。Mapper 注册到边缘的 **MapperRegistry**，按 **deviceName** 路由设备消息。v0.1.0 内置两个 Mapper（modbus 需显式启用，见下）：

### 6.3.1 Mapper 装配开关（v0.2.0）

v0.2.0 起，Mapper 装配由开关 **EDGEFLOW\_EDGECORE\_ENABLE\_MAPPER** 控制，默认 **true**（与旧版本行为一致）：

- **true**（或 1/on/yes，大小写不敏感）：装配全部 Mapper，正常采集与指令执行；
- **false**（或 0/off/no）：**不注册任何 Mapper、不启动采集循环**；云端下发的设备指令仅更新影子 **Twin.Desired**，不下发到设备——即**纯影子模式**。

### 6.3.2 mock\_sensor：模拟传感器

内置模拟温湿度传感器，用于验证全链路，无需真实硬件：

- 设备名：**sensor-01**；
- 采集周期：每 2 秒波动一次温度/湿度；
- 数据面：MQTT（遥测主题 **devices/default/sensor-01/telemetry**）；
- 支持指令：如 **targetTemp**（目标温度）写入期望值。

### 6.3.3 Modbus TCP Mapper

通过 **goburrow/modbus** 客户端库（社区标准 Modbus 客户端）对接 Modbus TCP 设备：

- 设备名：**mb-sensor-01**（unit ID 1），采集温度/湿度保持寄存器；
- 自带 **modbussim** 模拟器，无需真实设备即可联调；
- **操作台账**（op-ledger）：每次读写操作落 SQLite 台账，可查设备 ID、方向（up/down）与时间，便于审计排障；
- 指令链路与 **mock\_sensor** 一致：云端下发 **DeviceCommand{deviceName:"mb-sensor-01",...}**，Mapper 按指令写寄存器。

**启用方式**：Modbus Mapper 为可选接入，必须显式设置环境变量 **EDGEFLOW\_MODBUS\_ADDR**（如 127.0.0.1:15020 指向本机模拟器，或 < 设备 IP>:502 指向真实 Modbus 设备）后启动 **edgecore**，该 Mapper 才会注册；未设置时 **mb-sensor-01** 不存在。

### 6.3.4 设备命名空间 (v0.2.0)

v0.2.0 起, 设备按**命名空间** (namespace) 分组, 路由键为 `namespace/deviceName`, 同名设备可在不同命名空间共存:

- **解析优先级(三级):**Mapper 的 `WithNamespace` 选项 > 环境变量 `EDGEFLOW_MODBUS_NAMESPACE` > 默认 `default`;
- **注册表隔离:** `MapperRegistry` 按命名空间隔离路由, 同名设备分属不同命名空间互不干扰;
- **指令路由:** `device-command` 请求的 `namespace` 参与路由, 命名空间不匹配任何 Mapper 时边缘拒绝 502;
- **采集汇入 (v0.3.0 修复):** 采集数据按 `mapper` 自身命名空间汇入影子, 多命名空间部署不会出现 `default/` 双条目错位。

### 6.3.5 OPC-UA 接入状态 (v0.3.0 起)

v0.3.0 交付 **OPC-UA 第一阶段**: UA Binary 协议栈核心 (`pkg/opcua`, 零第三方依赖, 纯标准库实现)。当前能力与边界如下, 接入真实 OPC-UA 设备排后续版本:

- **已交付:** UA Binary 编解码 (`NodeId` 全编码形式、`Variant`、`DataValue`、`DateTime` 等类型) 与 HEL/ACK 握手 (`Dial/DialTimeout/ReadMessage/WriteMessage`);
- **未实现:** 设备读写服务 (`Read/Write/Subscribe`)、`SecureChannel` (当前为裸传输, `ChannelId=0`)、节点模型、`Discovery`、`Sign/SignAndEncrypt` 安全策略——因此尚无 **OPC-UA Mapper** 接入, 设备接入链路未打通;
- **安全边界:** 仅支持 `SecurityPolicy None` (明文), 严禁暴露到不可信网络, 仅限可信隔离网络使用;
- **验证状态:** 未与 `open62541 / node-opcua` 等第三方栈做过互操作验证。

### 6.3.6 Mapper 的两种工作模式

Mapper 由 `edgecore` 装配时统一注册, 按 `EventBus` 是否可用分为两种模式:

**MQTT 模式** `EventBus` 连接成功: 遥测发布到 MQTT 主题、订阅设备指令主题, 数据面完整 (broker 掉线后 `paho` 自动重连、订阅自动恢复);

**纯本地模式** MQTT 不可用 (未装配/连接失败降级): 采集照常写入影子、本地指令照常执行, 仅 MQTT 数据面不可用; 云边通道 (`DeviceCommand/DeviceReport`) 不受影响。

两种模式对云端完全透明: 云端只看到影子视图, 不感知边缘内部数据面形态。这也是第 九 章「`EventBus` 连接失败不必停机」的机制基础。

## 6.4 EventBus 数据面与降级

边缘内部通过 **EventBus** (MQTT, 本机 1883) 承载设备数据面：设备/Mapper 之间的遥测与指令走 MQTT 主题，云边管理消息仍走 WebSocket 通道，二者互不干扰。

表 6.1: EventBus 主题约定 (QoS 1)

| 主题                              | 方向      | 用途        |
|---------------------------------|---------|-----------|
| devices/<ns>/<device>/telemetry | 设备 → 边缘 | 设备遥测上报    |
| devices/<ns>/<device>/command   | 边缘 → 设备 | 指令下发      |
| edgeflow/<module>/<event>       | 模块间     | 内部事件 (预留) |

**降级行为：**MQTT broker 不可用或连接超时（默认 5 秒）时，Mapper 自动降级**纯本地模式**——设备采集与本地指令照常工作，仅数据面 MQTT 发布/订阅不可用；云边通道 (DeviceCommand/DeviceReport) 不受影响。已建立连接后的掉线会自动重连 (QoS 1, 订阅自动恢复)；若为**启动时**连接失败而降级，则降级是装配期决策，broker 恢复后需重启 edgecore 才能启用 MQTT 数据面。

```
# 订阅查看 sensor-01 遥测 (本机 mosquitto broker)
mosquitto_sub -p 1883 -t 'devices/default/sensor-01/telemetry' -v
# 通过 MQTT 数据面直接下发指令
mosquitto_pub -p 1883 -t 'devices/default/sensor-01/command' \
-m '{"deviceName":"sensor-01","property":"targetTemp","value":23}'
```

## 6.5 设备状态查询

云端提供统一查询入口，返回设备的 **properties** 与 **desired**：

```
# 全部设备状态
curl http://127.0.0.1:8080/api/v1/devices
# 单节点设备状态
curl http://127.0.0.1:8080/api/v1/nodes/{nodeID}/devices
```

- 节点不存在 → 404；
- 节点存在但无设备 → 200 + 空 items (可无分支遍历)；
- 开启 Token 认证时需携带 **Authorization: Bearer <token>** 请求头 (见第 八 章)。

**持久化注 (v0.4.0 起)：**设备 **desired** (期望值) 随云端 etcd 写穿持久化，跨云端重启保留；上报属性 (**properties**) 在云端重启后短暂清空，由边缘**不超过 1 个上报周期内**自动补报自愈。

## 6.6 设备接入检查清单

接入一台新设备时，按以下顺序核对：

1. **数据面**：设备/Mapper 与 broker（或本地模式）连通，遥测主题有数据（`mosquitto_sub` 可观测）；
2. **影子**：edgecore 日志出现 Mapper 注册成功，影子 `properties` 有值；
3. **上报**：等一个上报周期（默认 30s），云端 `GET /api/v1/devices` 可见设备；
4. **指令**：下发一条 `device-command`，确认 200 + 设备属性变化 + 期望值写入云端。

第 7 章 升级与回滚

EdgeFlow 的升级/回滚由安装管理 CLI `keadm` 提供，作用在 `keadm` 生成的产物文件层面 (`edgecore.env / edgecore.service / install.sh`)，配合操作台账 (`ops-ledger`) 实现可审计、可恢复的升级流程。部署在 Kubernetes 上的云端组件则走 Helm 路径。两条路径互不替代，本章分别说明；此外，云端从 `v0.4.0` 起具备独立的分级持久化与多副本升级语义（见第 7.7 节）。

7.1 机制总览

表 7.1: 升级回滚机制一览

| 能力   | 说明                                                                                                        |
|------|-----------------------------------------------------------------------------------------------------------|
| 备份模型 | 升级前把产物快照备份到 <code>backups/</code> (含 <code>manifest.json</code> 与每文件 <code>sha256</code> )                |
| 台账模型 | 每次操作（含失败）追加写 <code>ops-ledger.jsonl</code> ，逐行 JSON                                                       |
| 版本标记 | <code>edgecore.env</code> 内版本标记行， <code>join</code> 写入、 <code>upgrade</code> 更新、 <code>rollback</code> 恢复 |
| 数据隔离 | 不触碰数据目录（SQLite 等）与用户文件                                                                                    |

**边界:** 本机制只保证离线产物可恢复；镜像 tag 变更、edgecore 二进制替换、节点侧 `install.sh` 的实际执行不在产物级机制内，需结合发布流程执行。

7.2 升级前准备

- 1. 记录当前产物基线（可选但推荐）：

```
sha256sum $OUT/edgecore.env $OUT/edgecore.service $OUT/install.sh > baseline.sha
```

- 2. 如需在台账中记录操作人，设置环境变量：

```
export KEADM_OPERATOR=alice
```

未设置时台账记录 `unknown`。

7.3 升级：keadm upgrade

```
# 升级到 v0.7.0 (先备份 → 更新版本标记 → 写台账)
keadm upgrade --version=v0.7.0 --output-dir=./keadm-out

# 演练模式：备份后模拟失败（产物不会被修改），用于演练回滚流程
keadm upgrade --version=v0.7.0 --simulate-failure --output-dir=./keadm-out
```

执行流程：

1. **校验版本格式**：必须为 `vX.Y.Z`（如 `v0.7.0`），非法或与当前版本相同 → 退出码 2，不产生备份；
2. **预检**：三个产物文件（`env/service/install.sh`）任一缺失 → 退出码 1；
3. **备份**：复制产物到 `backups/<id>/`（`<id>` 为到秒的时间戳，同秒追加 `-2`、`-3...`），写入 `manifest.json`（时间/版本/操作人/文件清单/sha256）；
4. **更新版本标记**：整行替换 `edgecore.env` 中的版本标记；
5. **写台账**：追加 `result=ok` 记录。

任一环节失败：退出码 1，台账记 `failed`，并提示「执行 `keadm rollback --latest` 可恢复」。备份目录只增不删，保留恢复路径与审计现场。

## 7.4 回滚：keadm rollback

```
# 取最新有效备份回滚
keadm rollback --latest --output-dir=./keadm-out

# 指定备份 id 回滚（id 见 backups/ 目录名或台账）
keadm rollback --backup=20260814-190701 --output-dir=./keadm-out
```

执行流程：

1. **参数校验**：`--backup` 与 `--latest` 互斥，必须二选一，违规退出码 2；
2. **定位备份**：`--latest` 取时间戳最新（含序号后缀）；`--backup=<id>` 精确匹配，不存在 → 退出码 1；
3. **完整性校验**：`manifest.json` 可解析、字段完整、清单内文件存在且 `sha256` 一致；校验失败 → 报错并 **保留备份目录**；
4. **恢复**：逐文件复制回输出目录，回读校验内容一致，并强制恢复权限（`env 0600 / service 0644 / install.sh 0755`）；
5. **写台账**：追加 `result=ok`（`from=` 当前版本，`to=` 备份版本）。

**失败兜底**：任何自动恢复不可用的场景，备份目录保留完整快照，错误提示中给出逐文件人工恢复命令，例如：

```
cp backups/<id>/edgecore.env      <output-dir>/edgecore.env
cp backups/<id>/edgecore.service  <output-dir>/edgecore.service
cp backups/<id>/install.sh        <output-dir>/install.sh
```

## 7.5 台账查询：keadm ops-ledger

```
# 最近 20 条（默认）
keadm ops-ledger --output-dir=./keadm-out
# 最近 N 条
keadm ops-ledger --limit=5 --output-dir=./keadm-out
```

输出为 JSON 数组，逐行 JSON 原样保留，机器可解析。记录字段：**ts** (RFC3339 时间)、**action** (upgrade/rollback)、**from/to** (版本)、**result** (ok/failed)、**operator** (操作人)、**note** (备份 id、失败原因等)。台账不存在时输出 [] 并以 0 退出；损坏行跳过并附警告。

## 7.6 Helm 路径（云端组件）

云端 cloudcore 若通过 Helm Chart (build/charts/edgeflow) 部署，升级与回滚走标准 Helm 流程：

```
# 升级 (--install 兼容首次安装)
helm upgrade --install edgeflow build/charts/edgeflow \
  --set cloudcore.env.EDGEFLOW_CLOUDCORE_TLS=on \
  --set cloudcore.env.EDGEFLOW_CLOUDCORE_CERT_DIR=/data/certs

# 回滚到上一个版本
helm rollback edgeflow <rev>
```

版本标记约定：keadm join 生成的 edgecore.env 首行注释下含版本标记行，与 keadm init 默认镜像 tag (edgeflow/cloudcore:v0.7.0) 对齐：

```
# keadm 产物版本：v0.7.0
```

旧版 join 产物无此标记时，upgrade 视当前版本为 unknown，更新时自动追加标记行（兼容升级路径）。rollback 恢复后标记行随文件内容一并还原。

## 7.7 云端（cloudcore）升级与回滚（v0.4.0 起语义）

v0.4.0 起，云端引入**分级持久化**（嵌入式 etcd），v0.5.0 起支持**外部 etcd 模式**，v0.6.0 起支持**真多活多副本**，v0.7.0 新增**模型仓库键空间**——云端升级/回滚因此具备独立语义。本节按版本逐项说明；完整迁移 runbook 见部署文档 DEPLOYMENT.md (§10.7/§10.8/§10.9)。

### 7.7.1 v0.4.0：嵌入式 etcd 持久化升级

- **自动建库，无迁移脚本**：EDGEFLOW\_CLOUDCORE\_ETCD\_ENABLED 默认 true，升级后首次启动自动创建嵌入式 etcd 库并从前置 Range 全量 Load 既有数据；旧节点在保留期内 (NODE\_RETENTION，默认 24h) 重新注册后立即可见，无需重新注册；
- **Helm 升级必须确保 PVC**：Chart 默认挂载 1Gi PVC 到 /data；若以 emptyDir 运行，持久化**名存实亡**，容器重启即清空；



- **embed 模式 replicaCount 必须为 1**：多副本各自内嵌 etcd 会形成脑裂，Chart 在渲染期以 `{{ fail }}` 守卫拒绝 `replicaCount>1`；
- 监控告警阈值需容忍「Pod/上报数据重启后短暂清空（不超过 1 个上报周期自愈）」窗口；cloudcore 资源占用上升（RSS 约 31–34MB），Helm 资源建议 `requests 256Mi / limits 1Gi`。

### 7.7.2 备份与恢复 runbook（简化版）

在线快照（推荐；嵌入式 etcd 运行中执行）：

```
etcdctl snapshot save --endpoints=http://127.0.0.1:12379 snapshot.db
```

离线拷贝（需先停止 cloudcore 进程，保证数据目录一致）：

```
# 停进程后执行
cp -a data/etcd data/etcd.bak
```

恢复（将恢复出的数据目录替换到 `ETCD_DATA_DIR` 后启动 cloudcore）：

```
etcdctl snapshot restore snapshot.db --data-dir=<全新目录>
```

**注意：文件拷贝 ≠ 有效备份**——运行中直接复制数据目录会得到不一致状态，必须走 `etcdctl snapshot`（在线）或停进程后 `cp -a`（离线）。外部 etcd 模式的备份恢复由外部集群侧负责（同为 `etcdctl` 流程，`endpoints` 指向外部集群）。

### 7.7.3 v0.5.0：外部 etcd 模式切换

设置 `EDGEFLOW_CLOUDCORE_ETCD_ENDPOINTS`（非空）即切换为**外部直连共享 etcd 集群**模式（跳过 embed，不建目录不占端口；embed 相关变量设置仅告警忽略）。切换方式二选一：

- **快照恢复迁移**：外部集群先 `etcdctl snapshot restore` 导入既有数据，再切换 `endpoints`；
- **零迁移自愈**：不迁移历史数据，节点重新注册后台账与 `Desired` 重建，上报数据不超过 1 个周期自愈（适用于可接受重建的场景）。

**外部集群建议配置**：3 节点奇数/同地域；quota 256MiB；auto-compaction 1h；etcd 侧最小权限角色（`readwrite /edgeflow/`）。明文护栏与 TLS/mTLS 配置见第 八 章。

### 7.7.4 v0.6.0：外部模式多副本（真多活）

v0.6.0 起外部模式支持 `replicaCount>1` active-active 多副本。扩容前置条件（三项齐备）：

- **同版本**：所有副本必须同一 cloudcore 版本（混合版本未验证，且 v0.5.0×v0.6.0 混跑会被旧版 GC 误删活节点）；
- **3 节点 quorum**：外部 etcd 集群至少 3 节点，容忍单点故障；

- **共享 endpoints**: 所有副本指向同一外部 etcd 集群。

扩容:

```
kubectl scale deployment edgeflow-cloudcore --replicas=2
```

多副本形态下判活由 etcd **租约**承担 (NODE\_LEASE\_TTL, 默认 300s), NodeController 停用; MULTI\_REPLICA 由 Chart 自动注入, /healthz 反映 etcd 连接状态 (失联超过 TTL 返回 503, K8s liveness 重启自愈)。

**升级/回滚纪律**: 无论升级还是回滚, 一律**全停再全起**——先 `kubectl scale --replicas=0`, 确认全部副本退出后, 再以新版本 `scale --replicas=1` (或期望副本数) 拉起。**禁止混合版本多副本混跑** (v0.5.0×v0.6.0 同连一集群会误删活节点; v0.6.0×v0.7.0 未验证, 建议同版本全量切换)。回滚到 v0.5.0 后可选用 `etcdctl del /edgeflow/registry/heartbeats --prefix` 清理残留心跳键 (可选, v0.5.0 不读写该前缀)。

### 7.7.5 v0.7.0: 模型仓库升级 (零迁移)

v0.7.0 新增 `/edgeflow/models/` 键空间 (模型/版本/发布/部署影子), 与既有键空间完全隔离, **升级零迁移**: 边缘节点零动作, 旧版 edgecore 直接可用; schemaVersion 不 bump。

回滚到 v0.6.0 后, `/edgeflow/models/` 残留键无害 (v0.6.0 不读写该前缀); 如需彻底清理 (可选):

```
etcdctl del /edgeflow/models --prefix
```

### 7.7.6 版本升级路径一览

表 7.2: 云端版本升级路径 (v0.1.0 → v0.7.0)

| 路径              | 说明                                                               |
|-----------------|------------------------------------------------------------------|
| v0.1.0 → v0.1.1 | 直接升级, 零破坏 (安全加固)                                                 |
| v0.1.1 → v0.2.0 | 零破坏: 契约只增不改, 新能力显式配置生效                                           |
| v0.2.0 → v0.3.0 | 无破坏性变更                                                           |
| v0.3.0 → v0.4.0 | 默认启用嵌入式 etcd (自动建库、无迁移脚本); Helm 必须带 PVC; embed replicaCount 必须 1 |
| v0.4.0 → v0.5.0 | 默认行为不变; 外部模式为显式选择 (设置 ENDPOINTS 即切换)                             |
| v0.5.0 → v0.6.0 | 零迁移动作 (键空间兼容); <b>禁止混合版本多副本混跑</b> , 升级/回滚全停再全起                   |
| v0.6.0 → v0.7.0 | 零迁移 (新前缀隔离); 建议同版本全量切换 (混合版本未验证)                                 |

## 7.8 端到端演练建议

```
keadm init --output-dir=$OUT
keadm join --cloudcore-ip=192.168.1.10 --token=abc123 \
          --node-id=edge-worker-01 --output-dir=$OUT
keadm upgrade --version=v0.7.0 --simulate-failure --output-dir=$OUT # 预期失败
    exit=1
keadm rollback --latest --output-dir=$OUT # 恢复
keadm upgrade --version=v0.7.0 --output-dir=$OUT # 真实升级
keadm rollback --latest --output-dir=$OUT # 真实回滚
sha256sum -c baseline.sha # 产物一致
keadm ops-ledger --output-dir=$OUT # 台账 4 条
```

注意：重复升级每次都会产生新备份（只增不删），长期使用请定期人工归档 **backups/** 目录。

云端（cloudcore）侧的升级演练与 runbook 见部署文档 **DEPLOYMENT.md §10.7.5**（外部模式迁移）、**§10.8.5**（多副本扩容）与 **§10.9.4**（模型仓库升级）；云端走 Helm 路径升级/回滚时，同样遵循第 7.7 节的全停再全起纪律。

## 第8章 安全与认证

EdgeFlow v0.7.0 的安全基线由四层构成：**云边通道 mTLS**（双向认证）、**管理 API Token 认证**（默认关闭）、**审计台账**（默认开启）与 **外部 etcd 安全**（v0.5.0 起，控制面存储）。本章按层说明启用方式与运维要点。

### 8.1 云边通道 mTLS

云边 WebSocket 通道默认是明文 `ws://`（仅限可信网络/本机验证）。生产环境必须启用 **mTLS**（双向 TLS）：云端验证边缘证书、边缘验证云端证书与主机名，任何一侧校验失败连接在协议层之前被拒绝。

#### 8.1.1 启用开关

表 8.1: mTLS 启用开关

| 组件          | 开关 (=on)               | 证书目录变量 (默认)                                  |
|-------------|------------------------|----------------------------------------------|
| 云端 CloudHub | EDGEFLOW_CLOUDCORE_TLS | EDGEFLOW_CLOUDCORE_CERT_DIR<br>(data/certs/) |
| 边缘 EdgeHub  | EDGEFLOW_EDGECORE_TLS  | EDGEFLOW_EDGECORE_CERT_DIR<br>(data/certs/)  |

**两侧必须同时开启：**云端只开 TLS 会拒绝明文连接；边缘只开 TLS 无法通过明文云端的握手。边缘开启后连接地址自动归一化为 `wss://`。

```
# 云端
export EDGEFLOW_CLOUDCORE_TLS=on
export EDGEFLOW_CLOUDCORE_CERT_DIR=/data/certs
# 跨主机/集群部署必须注入 SAN（逗号分隔），非法条目启动即报错（fail-fast）
export EDGEFLOW_CLOUDCORE_TLS_SAN="IP:10.0.0.5,DNS:cloudcore.edgeflow.svc"
bin/cloudcore

# 边缘
export EDGEFLOW_EDGECORE_TLS=on
export EDGEFLOW_EDGECORE_CERT_DIR=/data/certs
export EDGEFLOW_EDGECORE_CLOUD_ADDR=wss://10.0.0.5:10000/v1/edge
bin/edgecore
```

**关键约束：**边缘连接的地址 (`wss://<host>:<port>`) 必须落在服务端证书 SAN 内，否则握手失败。未设置 SAN 时证书仅覆盖本机回环 (`127.0.0.1/localhost/cloudcore`)，mTLS 默认只在本机可用。

### 8.1.2 使用 keadm 生成 TLS 产物

```
# 云端产物：注入 TLS 开关 + 证书目录 + SAN
keadm init --tls --tls-san=IP:192.168.1.10 --output-dir=./keadm-out

# 边缘产物：注入 EDGEFLOW_EDGECORE_TLS=on + CERT_DIR，地址用 wss://
keadm join --cloudcore-ip=192.168.1.10 --cloudcore-port=31000 \
  --token=<token> --node-id=edge-worker-01 --tls --output-dir=./keadm-out
```

### 8.1.3 设备接入令牌认证

边缘节点注册接入的令牌校验（设备认证）**已实现**，链路如下：

- keadm join 生成 edgecore.env 时写入 EDGEFLOW\_EDGECORE\_TOKEN（值为 --token 参数）；
- edgecore 启动后随 Register 消息**携带**该令牌上报云端；
- 云端设置 EDGEFLOW\_CLOUDCORE\_NODE\_TOKEN 后启用校验：按**常数时间比较**，不匹配 → **拒绝注册**；
- 云端未设置该变量 → 不校验，保持向后兼容（默认空 = 不校验）。

```
# 云端：设置后开启设备接入校验（建议生产启用）
export EDGEFLOW_CLOUDCORE_NODE_TOKEN=<secret>
bin/cloudcore
```

## 8.2 证书管理

### 8.2.1 证书布局

证书目录（云端与边缘各自 data/certs/）包含：

私钥权限 0600，证书文件 0644。证书在组件首次以 TLS 启动时**自动生成**（幂等：已存在则加载校验，绝不重新生成）；也可用 hack/gen-certs.sh 预置。证书损坏/不完整 → **启动失败**（fail-fast，不降级）。

### 8.2.2 证书轮换与吊销

人工编排路径：

1. 签发新证书（先备份并删除旧文件，或使用新证书目录）；
2. 滚动重启组件，**先云后边**：云端加载新服务端证书，边缘重连时自动携带新客户端证书；
3. 验证新连接建立后，清除旧证书文件。

表 8.2: 证书文件布局

| 文件                            | 用途                    | 有效期           | 关键属性                                                                                        |
|-------------------------------|-----------------------|---------------|---------------------------------------------------------------------------------------------|
| ca.crt / ca.key               | 自签 CA                 | 10 年          | CN=edgeflow<br>Cert-Sign<br>SAN: 回环 + 注入值, EKU server-Auth<br>CN=edgeflow<br>EKU clientAuth |
| cloudcore.crt / cloudcore.key | 云服务端证书                | 1 年           | mTLS 握手校验依据与 crl.pem 对账自愈, 唯一事实源写路径                                                         |
| edgecore.crt / edgecore.key   | 边客户端证书                | 1 年           |                                                                                             |
| crl.pem                       | 吊销列表 (X.509 CRL 签名产物) | 7 天 (有新吊销时重签) |                                                                                             |
| crl.json                      | 吊销序列号来源记录             | —             |                                                                                             |
| crl.lock                      | 吊销写路径文件锁              | —             |                                                                                             |

轮换 CA 需**全量重签**所有叶子证书（旧叶子证书随 CA 更换立即失效）。CA 私钥仅靠文件权限保护，生产建议离线签发后移出节点或接入 KMS。

**自动轮换 (keadm cert rotate):** keadm cert rotate --node=<CN> --cert-dir=< 目录 > 按 CN 定位证书并 **事务化重签** (SAN 继承)，旧证书备份到 backups/< 时间戳 >/ (含 manifest.json)，输出新证书路径；随后需将新证书 **重新分发**到目标节点并重启生效。

**吊销(keadm cert revoke):** keadm cert revoke --node=<CN>|--serial=<hex> --cert-dir=< 目录 >

- --node 与 --serial **二选一互斥**：--node 按 CN 定位取序列号；--serial 接受十六进制（可带 0x 前缀，大小写均可，自动归一化），**不依赖证书文件**——可直接吊销已轮换的历史序列号（防泄露）；
- 序列号写入 `crl.json` 并重签 `crl.pem`；重复吊销 **幂等**（无副作用）；
- 吊销**不删除证书文件**：已分发的证书由对端在 mTLS 握手时按 CRL **拒绝**(cloudcore/edgecore 双侧消费端均接线，真实握手测试覆盖)；
- 吊销后需 keadm cert rotate 重签新证书并**重新分发**。

**吊销语义要点：**

- CRL 缺失（无 `crl.pem`）视为**无吊销**（向后兼容放行）；
- CRL 仅在发生新吊销时重生成（NextUpdate = 生成时 +7 天）；过期后**已吊销序列号仍被拒绝**（吊销检查先于过期检查），未吊销证书默认**放行**；FailOnExpired 为库级选项，当前 **未接线**到 mTLS 握手；
- `crl.json/crl.pem` 损坏或签名无效 → **拒绝**（fail-closed，不静默放行）；
- 吊销写路径文件锁 `crl.lock` 获取失败时自动**降级为无锁校验**（功能语义不变，仅损失 `crl.json` 领先时的即时重生成自愈），并输出 **5 分钟限频**的 Warn 告警日志，便于发现证书目录权限异常；
- 吊销**只增不减**：误吊销后无撤销命令，只能重新签发新证书（新序列号）；**请勿手工编辑 `crl.json`**（下次吊销时会按记录精确重生成 CRL，手工删除的条目将材料化生效）。

### 8.2.3 在线吊销查询 (OCSP)

云端提供标准 OCSP responder (RFC 6960): POST /ocsp。

- 请求体：DER 编码 OCSPRequest (Content-Type: application/ocsp-request)；响应体：DER 编码 OCSPResponse (application/ocsp-response)，由 **CA 私钥签名**；
- 吊销状态直接读取 `crl.json`，与 CRL **同源**——吊销后双通道（CRL 握手校验 + OCSP 在线查询）同时生效；

- 免认证端点, 以 per-IP 令牌桶限流防滥用: 默认 10 req/s、burst 20, 超限返回 429; 速率可经环境变量 `EDGEFLOW_CLOUDCORE_OCSP_RATE_LIMIT` 调整; 成功响应带 `Cache-Control: max-age=3600 (nextUpdate 7 天)`;
- 状态码: 200 = 签名响应; 400 = 请求非法/超限 (请求体上限 **16 KiB**, 空请求/DER 损坏/超限); 429 = 触发 per-IP 限流; 500 = CA 不可用/吊销记录损坏/响应构建失败 (fail-closed, 不静默放行);
- 客户端查询库函数 `OCSPStatus/OCSPStatusAt` 可用 (验签 + responderID + CertID 匹配), 新增新鲜度校验入口 `OCSPStatusAtWithPolicy/ParseOCSPResponseWithFreshness` (fail-closed: 过期/未来时间拒绝, 默认 5 分钟 skew); 当前均 **未接入**任何命令或握手路径——mTLS 握手消费的是 CRL 而非 OCSP (生产路径接入时须用 `WithPolicy` 入口)。

**安全边界:** `/ocsp` 是唯一免 **Token 认证** 的协议端点 (响应由 CA 私钥签名, 客户端必须验签防伪造)。该端点已内置 per-IP 限流 (默认 10 req/s、burst 20, 超限 429), 但仍监听全部接口, 建议置于受信内网/防火墙之后 (per-IP 粒度对分布式多 IP 放大不设防, 生产可叠加 LB 层全局限流); 吊销状态属公开信息 (协议设计如此), 不涉及凭据泄露。

### 8.3 管理 API Token 认证

管理 API (REST, 8080 端口) 默认**不认证** (向后兼容)。启用方式:

```
export EDGEFLOW_CLOUDCORE_AUTH=on
export EDGEFLOW_CLOUDCORE_API_TOKEN=<你的令牌>    # 必填, 未设置则启动失败
bin/cloudcore
```

- `EDGEFLOW_CLOUDCORE_AUTH=on` 且 `API_TOKEN` 未设置 → **启动失败** (fail-fast, 防止“开了认证却没令牌”的空转);
- 请求必须携带 `Authorization: Bearer <token>`;
- 缺失/非 Bearer 方案/令牌不匹配 → 401 + `WWW-Authenticate: Bearer` 响应头;
- 常量时间比较防时序侧信道; 认证通过者记为身份 `token` (单令牌模型, 持有令牌即管理员; 完整 RBAC 角色模型为后续版本)。

**模型 API 自动覆盖 (v0.7.0):** v0.7.0 新增的 17 个模型 API (模型/版本/发布/部署影子, 见第 4 章) 注册于同一 `apiMux`, `EDGEFLOW_CLOUDCORE_AUTH=on` 时**自动要求** `Authorization: Bearer <token>` (缺失/不匹配 → 401 fail-fast), 审计台账自动记录——无需任何额外配置。

### 8.4 外部 etcd 安全 (v0.5.0 起)

云端持久化存储可选择外部 etcd 集群 (`EDGEFLOW_CLOUDCORE_ETCD_ENDPOINTS` 非空即启用, 配置与迁移见第 2/7 章)。etcd 存储节点注册台账、设备 Desired 与模型仓库数据, 属于**控制面敏感数据**, 其安全配置与云边 mTLS、管理 API Token 是**两套独立体系, 互不替代**: 云边通道与 API 的认证不会自动保护 etcd 链路, 反之亦然。



#### 8.4.1 明文护栏（默认拒绝）

- 端点含非回环地址（非 127.0.0.1/localhost）且 未配置 TLS → cloudcore 拒绝启动 (fail-fast)，防止控制面数据明文暴露在网络上；
- 逃生门：EDGEFLOW\_CLOUDCORE\_ETCD\_ALLOW\_INSECURE=1 可放行明文（启动期输出大告警）。仅限可信内网使用，生产严禁开启。

#### 8.4.2 TLS 与 mTLS

- EDGEFLOW\_CLOUDCORE\_ETCD\_TLS\_CA 非空即启用 TLS：全部端点必须为 https://（任一 http 端点 → 启动失败）；
- ETCD\_TLS\_CERT 与 ETCD\_TLS\_KEY 同设即 mTLS（双向认证）；只设其一 → 启动失败 (fail-fast)；
- 最低 TLS 版本 TLS 1.2；
- 启动时执行连通性检查（线性一致读 schemaVersion 键，至多 3 次 × 5s + 1s 间隔，约 17s）：探活失败 / 鉴权被拒 / 明文被护栏拦截 → 拒绝启动，日志区分 Unavailable（网络/集群不可达）与 PermissionDenied（凭据/角色不足）两类文案。

#### 8.4.3 etcd 侧最小权限

- 为 cloudcore 创建最小权限角色：仅 readwrite /edgeflow/ 前缀键空间，不建议授予全库 root；
- 生产建议 etcd 开启 --client-cert-auth（客户端证书认证）配合 mTLS 使用；
- 版本限制：v0.5.0 起不支持 etcd 鉴权参数透传（用户名/密码经环境变量传入，见 KNOWN-ISSUES L1，规划后续版本），鉴权凭据通过 etcd 侧证书体系解决。

### 8.5 审计台账

管理 API 的所有操作默认写入审计台账：

- 路径：EDGEFLOW\_CLOUDCORE\_AUDIT\_PATH，默认 data/audit-ledger.jsonl；
- 格式：JSONL（逐行 JSON，追加写，不覆盖历史）；
- 记录内容：请求路径、方法、结果、身份（Token 认证下含 operator；认证失败记录 anonymous）；
- 审计失败不阻断 API：写盘失败只记错误日志，业务请求照常处理（可用性优先）。

```
# 查看审计台账（每行一条记录）
tail -f data/audit-ledger.jsonl
```

## 8.6 安全基线小结

- 已实现:双向认证(云验边、边验云 + 主机名)、设备接入令牌认证(云端设 `EDGEFLOW_CLOUDCORE_NODE_TOKEN` 后启用,见「设备接入令牌认证」小节)、私钥 0600、强制 TLS 1.2+、证书损坏 fail-fast、Token 认证(默认关, v0.7.0 起模型 API 自动覆盖)、审计台账(默认开)、证书吊销(CRL 离线 + OCSP 在线,见「证书轮换与吊销」与「在线吊销查询(OCSP)」小节)、外部 etcd 明文护栏与 TLS/mTLS (v0.5.0,见「外部 etcd 安全」小节);
- 已知限制: CSR 审批流未实现;证书 CN 与 nodeID 未绑定校验;单令牌模型(无多角色授权);外部 etcd 不支持鉴权参数透传(凭据经证书体系解决)。

# 第 9 章 常见问题与故障排查

本章汇集 v0.7.0 实测与文档沉淀的常见问题。排障第一步永远是**看日志**，先定位故障在哪一层：云边通道、容器运行时、设备数据面，还是 API 层。

## 9.1 日志位置

表 9.1: 各组件日志位置

| 组件                 | 日志位置                                  |
|--------------------|---------------------------------------|
| cloudcore          | stdout / 日志文件（部署形态而定）                 |
| edgecore (systemd) | journalctl -u edgecore -e / -f 跟踪     |
| edgecore (裸进程)     | stdout                                |
| 审计台账               | data/audit-ledger.jsonl (云端)          |
| 操作台账               | <output-dir>/ops-ledger.jsonl (keadm) |

## 9.2 常见问题速查表

## 9.3 分层排查指引

### 9.3.1 云边通道层

1. 确认 edgecore 进程存活且已注册：

```
curl http://127.0.0.1:8080/api/v1/nodes
```

2. 节点存在但状态 **Offline**：心跳停滞超过 180s（约 6 个 30s 心跳周期；**外部多副本形态**判活为 etcd 租约，阈值取 EDGEFLOW\_CLOUDCORE\_NODE\_LEASE\_TTL（默认 300s），见第 5 章）。检查网络、云端进程；
3. 节点不存在（404）：从未注册或注册失败。看 edgecore 日志中 Register/RegisterAck 是否成功；
4. 启用 mTLS 后连不上：先确认两侧开关都开、证书目录正确、边缘连接地址在服务端证书 SAN 内（见第 八 章）。

### 9.3.2 容器运行层

1. 调谐周期默认 5s，改动 Pod 定义后最多等一个周期生效；
2. 容器反复拉起并进入退避：区分「业务崩溃」与「退出型镜像」。CrashLoopBackOff 是保护机制，不是故障本身；

表 9.2: 常见问题与处置

| 现象            | 常见原因                  |
|---------------|-----------------------|
| edgecore 起不来  | 网络不通 / 配置错误           |
| EventBus 连接失败 | MQTT broker 未启动 / 地址错 |

3. 镜像漂移检测会在期望镜像与运行镜像不一致时自动重建（每轮 1 个，逐轮滚动），滚动期间短暂重启属预期。

### 9.3.3 设备数据面层

1. GET /api/v1/devices 看不到设备：确认 edgecore 内 Mapper 已注册（日志有「Mapper 注册完成」），MQTT 模式下确认 broker 可达；
2. 设备属性不更新：确认采集周期（mock\_sensor 默认 2s）与上报周期（默认 30s）；
3. 指令下发失败：按状态码区分——404 未送达（节点离线，修通道）、502 送达被拒（改请求）、504 未确认（重试）。

## 9.4 API 错误语义速记

- 404：节点未注册/离线，**无需重试**，先修通道；
- 502：消息已送达但边缘拒绝，**重试无意义**，检查请求内容；
- 504：可能送达但未确认，**可重试**，边缘幂等去重；
- 错误响应统一为 JSON：{"error": "< 机器可读原因 >"}。

完整语义见附录 B。

## 9.5 环境相关的已知边界

- **macOS/Docker Desktop**：宿主机无法直接路由 kind 节点 IP 的 NodePort（实测 connection reset），用 `kubect1 port-forward` 替代；Linux 上可直接 `nodeIP:nodePort` 访问；
- **真实多节点集群**：v0.1.0 验证范围为 kind 单节点集群 + 单边缘节点（2026-08-14 实测），多节点（10+）E2E 与压测未做；
- **镜像仓库**：镜像未推送远端仓库，跨机部署需自建 registry（`docker run -d -p 127.0.0.1:5001:5000 registry:2 + buildx` 双架构构建，见 docs/MULTIARCH.md）；
- **nodeID 字符约束**（v0.4.0 起）：节点 ID 必须匹配正则 `^[A-Za-z0-9._]+$`，含 / 的 nodeID 拒绝写入云端台账。

## 第 A 章 附录 A 环境变量参考

以下为 EdgeFlow v0.7.0 的主要环境变量（含组件装配与运维所需全部变量；Modbus 组独立成节见下）。设置方式：启动进程前 `export`，或由 `keadm join` 生成的 `edgecore.env` 统一注入。

### A.1 cloudcore 组

附注：

- 外部模式(ETCD\_ENDPOINTS 非空)下,嵌入式 etcd 变量(ETCD\_DATA\_DIR/ETCD\_CLIENT\_URL/ETCD\_PEER\_URL/ETCD\_QUOTA\_BACKEND\_BYTES/ ETCD\_AUTO\_COMPACTION\_\*) **不生效**, 设置仅输出 Warn;
- 外部模式下 NODE\_TIMEOUT 不再作为判活阈值、NODE\_SCAN\_INTERVAL 迁用为重扫/GC 周期（见第 5 章）;
- NODE\_LEASE\_TTL/RELEASE\_LOCK\_TTL 仅外部模式消费, embed/纯内存显式设置仅 Warn 忽略。

### A.2 edgecore 组

### A.3 Modbus 组

时长类变量（调谐/上报周期等）合法范围 **1s 至 10m**，超出或非法回退默认值并告警；v0.3.0 起启动日志对周期类 env **三态明示**（合法/ 越界回落/未设置）。

EDGEFLOW\_\* 环境变量共 **49 个**：cloudcore 组 30 个 + edgecore 组 16 个 + Modbus 组 2 个 + 测试专用 EDGEFLOW\_TEST\_DUR 1 个（仅测试代码使用，不影响部署）；另有可选模拟器变量 EDGEFLOW\_MODBUS\_SIM\_PORT（不计入总数）与 keadm/开发组 5 个（见下节）。

### A.4 keadm / 开发脚本组

后四个变量仅用于开发环境脚本 `hack/dev-up.sh / dev-down.sh`，不影响生产部署。

表 A.1: cloudcore 环境变量

| 变量                            | 默认值         | 说明                         |
|-------------------------------|-------------|----------------------------|
| EDGEFLOW_CLOUDCORE_PORT       | 8080        | 管理 API 监听端口                |
| EDGEFLOW_CLOUDCORE_HUB_PORT   | 10000       | CloudHub (云边通道) 端口         |
| EDGEFLOW_CLOUDCORE_TLS        | 关           | on 启用 mTLS                 |
| EDGEFLOW_CLOUDCORE_CERT_DIR   | data/certs/ | 证书目录                       |
| EDGEFLOW_CLOUDCORE_TLS_SAN    | 空（仅回环）      | 证书 SAN, 逗号分隔, 非法 fail-fast |
| EDGEFLOW_CLOUDCORE_NODE_TOKEN | 空           | 设备接入令牌: 设                  |

表 A.2: edgecore 环境变量

| 变量                                       | 默认值                          | 说明                            |
|------------------------------------------|------------------------------|-------------------------------|
| EDGEFLOW_EDGECORE_NODE_ID                | edge-< 主机名 >                 | 节点 ID                         |
| EDGEFLOW_EDGECORE_CLOUD_ADDR             | ws://127.0.0.1:10000/v1/edge | 云端 Cloud-Hub 地址               |
| EDGEFLOW_EDGECORE_TLS                    | 关                            | on 启用 mTLS (wss://)           |
| EDGEFLOW_EDGECORE_CERT_DIR               | data/certs/                  | 证书目录                          |
| EDGEFLOW_EDGECORE_DB_PATH                | 内置默认                         | MetaManager SQLite 路径         |
| EDGEFLOW_EDGECORE_MQTT_ADDR              | tcp://127.0.0.1:1883         | 本机 MQTT broker 地址             |
| EDGEFLOW_EDGECORE_RECONCILE_INTERVAL     | 5s                           | Edged 调谐周期                    |
| EDGEFLOW_EDGECORE_REPORT_INTERVAL        | 30s                          | Pod 状态上报周期                    |
| EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL | 30s                          | 设备状态上报周期                      |
| EDGEFLOW_EDGECORE_MQTT_CONNECT_TIMEOUT   | 5s                           | 单次 MQTT 建连超时                  |
| EDGEFLOW_EDGECORE_TOKEN                  | 空                            | 已消费: keadm join 写入 (--token), |



表 A.3: Modbus Mapper 环境变量

| 变量                        | 默认值     | 说明                                                                                                         |
|---------------------------|---------|------------------------------------------------------------------------------------------------------------|
| EDGEFLOW_MODBUS_ADDR      | 空       | Modbus TCP 从站地址 ( <code>host:port</code> ); 空 = 不启用 Modbus Mapper                                          |
| EDGEFLOW_MODBUS_NAMESPACE | default | 设备命名空间: 三级解析 (WithNamespace 选项 > env > default); 注册表按 ns 隔离, 同名设备分 ns 共存, 指令按 ns 路由 (错误 ns → 502) (v0.2.0) |
| EDGEFLOW_MODBUS_SIM_PORT  | 15020   | 模拟从站监听端口 (可选, 仅开发/演示环境)                                                                                    |

表 A.4: keadm 与开发脚本环境变量

| 变量                    | 默认值          | 说明                                            |
|-----------------------|--------------|-----------------------------------------------|
| KEADM_OPERATOR        | unknown      | keadm 操作台账记录的操作人                              |
| EDGEFLOW_CLUSTER_NAME | edgeflow-dev | 开发脚本 kind 集群名 ( <code>hack/dev-up.sh</code> ) |
| EDGEFLOW_EDGE_NODES   | 1            | 开发脚本边缘模拟容器数量                                  |
| EDGEFLOW_EDGE_IMAGE   | alpine:3.20  | 开发脚本边缘容器镜像                                    |
| EDGEFLOW_EDGE_ARGS    | 空            | 开发脚本追加给 edgecore 的参数                          |

## 第 B 章 附录 B API 状态码语义

管理 API (REST, 默认 8080) 状态码语义:

**记忆要点:** 404 = 没送达; 502 = 送达但被拒; 504 = 可能送达但未确认; 409 = 送达但超卖/冲突拒绝; 202 = 已受理异步执行; 429 = 限流。错误响应统一为 JSON: `{"error": "<机器可读原因 >", ...}`。

表 B.1: API 状态码语义

| 状态码 | 含义                                                                            |
|-----|-------------------------------------------------------------------------------|
| 200 | 成功；下发类接口表示 <b>边缘已确认</b> （Ack ok），响应 <code>{"status":"ok","acked":true}</code> |
| 202 | 已受理，异步执行：模型发布创建/回滚置位，结果以 <b>release</b> 对象回读为准（v0.7.0）                        |
| 400 | 请求非法：JSON 解析失败 / 缺必填字段 / operation 或 kind 不在白名单                               |
| 401 | 未授权：Token 认证开启且请求未携带/携带错误令牌（附 <b>WWW-Authenticate: Bearer</b> ）               |
| 404 | 节点未注册或离线（ <b>ErrNodeOffline</b> ）；单资源查询不存在                                    |
| 409 | 冲突：podsync 资源超卖拒绝 <b>EDGEFLOW_RESOURCE_EXHAUSTED</b> （不落盘不建容器，v0.2.0）；模型      |

## 第 C 章 附录 C 术语表

**CloudCore (云端核心)** 云端主进程, 含 CloudHub (云边通道服务端)、NodeController (节点健康扫描)、etcdstore (v0.4.0 嵌入式 / v0.5.0 外部直连持久化存储)、modelrepo/modelrelease (v0.7.0 模型仓库与灰度发布控制器) 与 REST API。

**EdgeCore (边缘核心)** 边缘节点常驻进程, 含 EdgeHub (云边通道客户端)、Edged、MetaManager、DeviceTwin、Mapper 框架与 EventBus。

**Edged** 边缘容器运行时管理模块: 声明式调谐 (默认 5s 周期)、多副本、健康自愈与镜像漂移检测。

**MetaManager** 边缘元数据管理: KV + 节点 + Pod 数据持久化到本机 SQLite (WAL), 断网自治的数据基础。

**DeviceTwin (设备影子)** 每台设备的数字孪生: **properties** (实际值) + **desired** (期望值)。

**Mapper (设备映射器)** 把具体设备协议翻译为统一设备模型的组件; v0.1.0 内置 mock\_sensor 与 Modbus TCP 两个 Mapper。

**PodSync** 云端向边缘可靠下发 Pod 配置的链路 (add/update/delete)。

**ConfigSync** 云端向边缘可靠下发 ConfigMap/Secret 配置的链路。

**NodeJob** 云端任务管理 (任务分发 CRD): **已决策关闭** (v0.1.0 范围外, 协议占位标注)。

**嵌入式 etcd** 云端默认持久化形态 (v0.4.0): cloudcore 内嵌单成员 etcd (127.0.0.1:12379/12380, 仅绑回环), 写穿持久化节点注册台账与设备 Desired; Pod 状态与上报属性不落盘 (重启短暂清空自愈)。

**外部 etcd 模式** 设置 EDGEFLOW\_CLOUDCORE\_ETCD\_ENDPOINTS 后直连共享 etcd 集群 (v0.5.0), 支持 TLS/mTLS、明文护栏, v0.6.0 起支持多副本真多活。

**写穿 (write-through)** 写入先落 etcd 成功再更新内存缓存; “写成功 = 已持久化” (v0.4.0 起)。

**真多活 / 租约判活** 外部多副本形态 (v0.6.0): 心跳落盘为 etcd 租约 (NODE\_LEASE\_TTL 默认 300s), 判活 = 心跳键存在性 (Ready/Offline/不存在 + 瞬时 Unknown), 配 GuardedDelete 删除守卫与 CAS 写。

**模型仓库** 云端模型与版本台账 (v0.7.0): Model (name 唯一) + ModelVersion (镜像即模型、Tag 即版本, 状态机 draft→active→archived)。

**灰度发布** 模型版本按白名单/百分比分批下发 (v0.7.0): 创建返回 202 异步受理, 批内串行、批间可暂停, 支持取消与回滚; 领跑锁保证多副本单执行者。

**部署影子** 模型发布在节点上的落点记录 `/edgeflow/models/deployments/<model>/<nodeID>` (v0.7.0, podsync + config-sync 双 acked 后写穿)。

**领跑锁** 发布执行权锁 (grant-per-claim, TTL 60s 可续): 多副本外部模式下保证同一发布仅一个执行者, 崩溃 TTL 接管续跑。

**mTLS** 双向 TLS: 云端验证边缘证书、边缘验证云端证书与主机名, 云边通道默认实现。

**影子设备** 云端视角的设备逻辑实体, 由边缘影子周期上报汇聚而成; v0.4.0 起 Desired 云端持久化 (写穿 etcd), properties 仍为内存态 (重启 1 上报周期自愈)。

**EventBus** 边缘内部 MQTT 设备通信总线 (数据面), broker 不可用时 Mapper 降级纯本地模式。

**op-ledger** Modbus Mapper 的读写操作台账 (SQLite), 记录设备 ID、方向与时间。

**ops-ledger** keadm 升级/回滚操作台账 (`ops-ledger.jsonl`)。

## 第 D 章 附录 D 版本与变更记录

### D.1 v0.1.0 (2026-08-14, MVP 首发)

- 发布内容: 云边通信 (WebSocket 注册/心跳/指数退避重连/可靠投递)、边缘自治 (Edged 声明式调谐/多副本/健康自愈/镜像漂移检测/断网自治)、设备管理 (DeviceTwin/Mapper/EventBus/Modbus)、生产加固 (mTLS/多架构镜像/Helm Chart/keadm 安装 CLI/升级回滚)、发布文档 (API 规范/部署指南/示例);
- 质量门: 24 个包 `go test -race` 全绿; 总覆盖率 **77.8%** (门槛 70%); `golangci-lint` 0 issues; P0/P1 审查发现 0; 端到端示例 `examples/demo.sh` 通过 3 次;
- 基线说明: 以上质量门数据为 **v0.1.0 发布时基线** (2026-08-14); 8-15/8-16 功能批次 (吊销闭环、OCSP、契约测试、OpenAPI 语义等) 与 8-18 v0.1.1 安全加固轮 (/ocsp 限流与缓存、OCSP 新鲜度校验、CRL 锁降级) 后, 全仓库测试函数 **623 个** (81 个测试文件), 其中 `pkg/certs` 74 个 (CRL/OCSP 吊销核心);
- 制品: `release/v0.1.0/` (6 二进制 + Chart 包 + checksums.txt + sbom.json + images.json); 镜像 `edgeflow/cloudcore:v0.1.0` / `edgeflow/edgecore:v0.1.0` (linux/amd64 + arm64)。

### D.2 v0.1.1 (2026-08-18, 安全加固)

- /ocsp per-IP 限流 (默认 10 req/s、burst 20, 超限 429) + 成功响应 `Cache-Control: max-age=3600`;
- OCSP 客户端库新增新鲜度校验入口 (fail-closed, 默认 5min skew; 生产握手路径未接线, mTLS 仍走 CRL);
- CRL 锁降级无锁校验 + 5min 限频 Warn; P2 收尾 (WriteTimeout、Route 注册带 namespace、LastReportedAt 单调保护等);
- 新增 `EDGEFLOW_CLOUDCORE_OCSP_RATE_LIMIT`; 错误码表增 429; v0.1.0 → v0.1.1 直接升级, 零破坏。

### D.3 v0.2.0 (2026-08-18, 资源调度 + Mapper 开关 + Modbus 命名空间)

- podsync 支持 K8s 风格 `resources` (cpu/memory request/limit, 零值 = 不限制): 格式非法/request>limit → 400, 超卖拒绝 → **409 EDGEFLOW\_RESOURCE\_EXHAUSTED** (不落盘不建容器); 容器落地 `--cpus/--memory` (swap 禁用); **资源漂移检测** (外部改 limit → 自动 stop+ 重建, 每轮最多 1 个); 节点容量自动探测 + env 覆盖, 超卖率默认 150%;

- Mapper 装配开关 `EDGEFLOW_EDGECORE_ENABLE_MAPPER`（默认 `true`；关闭 = 纯影子模式，不注册 Mapper、不采集）；
- Modbus 设备命名空间（`EDGEFLOW_MODBUS_NAMESPACE`，默认 `default`）：注册表按 ns 隔离，同名设备分 ns 共存，指令按 ns 路由（错误 `ns → 502`）；
- `edgecore` 组新增 5 个 env；错误码表增 409；云边契约只增不改，升级零破坏。

#### D.4 v0.3.0 (2026-08-19, KNOWN-ISSUES 闭环 + OPC-UA 协议栈)

- 四条已知问题闭环：采集汇入影子按 mapper 自身 ns、测试注入点、podsync 400 分支 JSON 结构化、周期 env 启动日志三态明示；
- **OPC-UA 第一阶段**：pkg/opcua UA Binary 协议栈核心（NodeId/Variant/DataValue 编解码 + HEL/ACK 握手，纯标准库零依赖）；**未实现**设备读写服务、SecureChannel、Discovery——仅 SecurityPolicy None **明文**，严禁暴露到不可信网络，未与第三方栈互操作验证；
- 无新增 env、无破坏性变更。

#### D.5 v0.4.0 (2026-08-24, 云端持久化：嵌入式 etcd)

- 云端三存储(registry/podstatus/devicestatus)由纯内存改为 **嵌入式单成员 etcd**(127.0.0.1:12379/12380. 只绑回环)**写穿持久化**:落盘 = 节点注册元数据 + 设备 Desired;不落盘 = 心跳/Status/Pod 状态/设备 Properties（重启 1 上报周期自愈）；
- 坏库默认降级纯内存 + 大告警,ETCD\_STRICT=1 fail-fast;GC 级联(节点删除 DeleteRange 设备子树, CleanupLoop 1h)；
- **升级注意**：默认启用（无迁移脚本）；Helm 必须带 PVC（默认 1Gi）；**embed 模式 replicaCount 必须 = 1**（多副本脑裂, Chart {{ fail }} 守卫）；备份恢复走 `etcdctl snapshot`，文件拷贝 有效备份；资源 requests 256Mi/limits 1Gi；
- nodeID 字符约束 `^[A-Za-z0-9._]+$`；cloudcore 组新增 9 个 env。

#### D.6 v0.5.0 (2026-08-24, 外部 etcd 模式)

- `EDGEFLOW_CLOUDCORE_ETCD_ENDPOINTS` 非空 = 外部直连共享 etcd 集群（clientv3, 跳过 embed，不建目录不占端口）；业务层零改动；
- **明文护栏**：非回环 + 无 TLS → 拒绝启动；逃生门 `ETCD_ALLOW_INSECURE=1`（仅可信内网）；TLS\_CA 启用 TLS1.2+，CERT/KEY 同设即 mTLS；启动连通性检查（17s）失败拒绝启动，区分 Unavailable/PermissionDenied 文案；
- 单写者形态铁律：两种模式 replicaCount 均必须 1（Chart {{ fail }} 守卫）；

- 默认行为不变（不设 ENDPOINTS = embed 逐位不变）；切换走部署指南 §10.7.5 迁移 runbook；建议外部集群 3 节点奇数/同地域/quota 256MiB/compaction 1h/最小权限角色（readwrite /edgeflow/）。

## D.7 v0.6.0（2026-08-25，真多活）

- 外部模式 **replicaCount > 1 active-active 放开**：心跳落盘为 etcd 租约（grant-per-heartbeat, NODE\_LEASE\_TTL 默认 300s）；判活三态（不存在/Ready/Offline + 瞬时 Unknown），事件源三路互兜（watch 增量/周期重扫/CloudHub 断开事件）；
- **GuardedDelete 守卫**（活租约拒绝删除）、watch 缓存同步、**SetDesired CAS**（冲突重试 3, HTTP 仍 200 + 日志 concurrent-write）；外部模式 NodeController 停用；
- /healthz 多副本绑定 etcd 连接（失联 >TTL → 503, liveness 重启自愈）；
- **etcd 故障 >TTL → 全量软离线**（有界，恢复 1 心跳周期自愈、零数据删除），与 v0.5.0 “判活不受存储故障影响”不同；
- **升级必须全停再全起（scale 0→1）**，禁止 v0.5.0 × v0.6.0 混跑（旧版 GC 误删活节点，L15）。

## D.8 v0.7.0（2026-08-25，模型仓库/版本管理/灰度发布）

- 云端新增 **modelrepo + modelrelease**：模型台账 Model、版本台账 ModelVersion(draft→active→archived)、部署影子 DeploymentState、灰度发布 ModelRelease（白名单/百分比二选一，创建返回 **202**，批内串行、支持取消/回滚，领跑锁保证单执行者）；
- **API 端点 14 → 31**（新增 17 个模型 API，全部挂既有 apiMux, AUTH=on 自动要求 Bearer Token）；错误码表增 **202/422**，409 家族扩展；
- 下发链路边缘零改动(podsync + config-sync, 命名 **edgeflow-model-<sanitized>**, namespace **edgeflow**, replicas=1)；
- 三模式兼容（纯内存 L22 重启丢失 / embed / 外部 CAS+guard+watch+ 领跑锁）；升级零迁移（新前缀 /edgeflow/models/），建议同版本全量切换（L29）。

## D.9 已实现 vs 即将上线 / 范围外



表 D.1: 能力对照

| 类别         | 内容                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 已实现        | 云边通信、边缘自治、设备管理、mTLS、Token 认证（默认关）、设备接入令牌认证、证书吊销（CRL 离线 + OCSP 在线）、云边通道 gzip 压缩（默认开，协商式）、审计台账、keadm (init/join/cert/upgrade/ rollback/ops-ledger/batch/reset/version)、批量与灰度 (keadm batch)、Helm Chart、多架构镜像、/metrics 可观测性、API 兼容矩阵 (docs/API-COMPATIBILITY.md + tests/contract)、镜像安全扫描 (Trivy 构建期 0 漏洞，发布流程级，见 docs/SECURITY-SCAN.md)、资源调度与超卖准入 (v0.2.0, K8s 风格 resources + 409 超卖拒绝 + 漂移重建)、Mapper 装配开关与设备命名空间 (v0.2.0)、OPC-UA 协议栈核心 (v0.3.0, 明文、仅限可信网络)、云端分级持久化 (v0.4.0 嵌入式 etcd 写穿)、外部 etcd 模式与明文护栏 (v0.5.0)、真多活多副本 (v0.6.0, 租约判活 + CAS + 删除守卫)、模型仓库/版本管理/ 灰度发布 (v0.7.0, 17 个模型 API, 端点 14→31) |
| 即将上线 / 范围外 | NodeJob 任务分发（已关闭）、完整 RBAC 角色模型（仅 Token 单令牌）、Flannel、OPC-UA 设备读写服务与 Mapper 接入（协议栈核心已实现，v0.3.0）、Protobuf 编码、训练平台/模型评测                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

## 第 E 章 附录 E 已知限制与边界

1. **云端分级持久化 (v0.4.0 起)**: 注册台账与设备 Desired 跨重启保留 (写穿 etcd); Pod 状态与上报属性重启后短暂清空 (**1 上报周期自愈, 非永久丢失**); 心跳/Status 不落盘 (重启待首心跳翻新); 在途未确认消息可能丢失, 由上层控制器恢复后重新下发。
2. **镜像未推送远端仓库**: 镜像仅本地/自建 registry 可用, GitHub 远端 CI(lint+test+release) 未在真实远端运行。
3. **GitHub remote 未配置**: 配置 SSH key 后 `git remote add origin` 推送即可启用远端 CI。
4. **真实多节点集群未验证**: v0.1.0 在 kind 单节点集群 + 单边缘节点跑通 (2026-08-14 实测); 真实多节点 (10+) E2E 与 100 节点压测未在真实集群执行; 本机**模拟压测基线** (10/100 节点并发注册/心跳, 100% 成功率) 已完成(hack/load-test, 见 docs/PERFORMANCE-BASELINE.md)。
5. **macOS 本机验证限制**: 宿主机无法直接路由 kind 节点 IP 的 NodePort (实测 connection reset), 需 `kubectrl port-forward`; mTLS 默认证书仅覆盖回环地址, 跨主机必须注入 SAN。
6. **keadm 产物级升级边界**: 备份/恢复只覆盖 `env/service/ install.sh` 三个产物文件, 不触碰数据目录与用户文件; 真实部署 (镜像 tag、edgecore 二进制) 升级需结合发布流程。
7. **证书管理**: CSR 审批流、证书 节点 CN 强绑定未实现; CA 私钥仅文件权限保护。(吊销已实现: CRL 离线 + OCSP 在线, 见第 8 章「证书轮换与吊销」「在线吊销查询 (OCSP)」小节。)
8. **手册更新方式**: 本手册 LaTeX 工程纳入 Git 仓库; 修改 `docs/manual/` 下章节文件后, 用 `xelatex` 编译主文件生成 PDF (ctexbook 文档类)。
9. **nodeID 字符约束 (v0.4.0)**: 节点 ID 必须匹配正则 `^[A-Za-z0-9._]+$`, 含 / 的 nodeID 拒绝写入云端台账。
10. **外部模式判活依赖 etcd 可用性 (L12)**: 外部多副本形态下 etcd 故障超过 `NODE_LEASE_TTL` → 全量软离线 (有界, 恢复 1 心跳周期自愈、零数据删除), 监控告警阈值按  $2 \times \text{TTL}$  折算。
11. **混合版本多副本禁止 (L15/L29)**: 升级/回滚一律全停再全起 (scale 0→1);  $v0.5.0 \times v0.6.0$  混跑会误删活节点 (旧版 GC 无 hb 视角),  $v0.6.0 \times v0.7.0$  混跑未验证, 建议同版本全量切换。
12. **纯内存模式模型发布重启丢失 (L22)**: `ETCD_ENABLED=false` 时模型发布任务与部署影子为内存态, 重启丢失; 生产建议 `embed/` 外部 etcd 模式。

13. **半部署状态 (L23)**: podsync 成功而 config-sync 失败的发布批次计 **failed**, 重试/回滚时按实际部署影子判定。
14. **回滚部分失败仍置 rolled\_back (L24)**: 回滚过程中节点失败不回滚中止, 状态机仍收敛为 **rolled\_back**, 需按部署影子核对残留节点。
15. **OPC-UA 协议栈明文 (KNOWN-ISSUES §3)**: v0.3.0 协议栈仅 SecurityPolicy None (明文), 未与第三方栈互操作验证; 严禁暴露到不可信网络, 仅限可信隔离网络。
16. **终态 release 键永久保留 (L31)**: 发布终态 (succeeded/ failed/canceled/rolled\_back) 键保留作审计痕迹, 不自动清理; 可选 `etcdctl del /edgeflow/models --prefix` 手工清理。
17. **无分页 (L28)**: 列表类 API (模型/版本/发布/部署影子) 暂不支持分页, 数据量大时响应体增长, 请按需控制台账规模。